

CS208 算法设计课件中文翻译

第1周 算法、证明技巧与应用 (Definitions, Proof Techniques, and Applications)

什么是算法？ (What Is an Algorithm?)

定义 算法是一种有限、精确、有效的过程，它接受输入，执行一系列明确定义的步骤，并产生输出来解决明确定义的计算问题。

程序是算法的实现，但不是算法本身。

有效算法的关键属性 (Key Properties of a Valid Algorithm)

具有明确执行规则的有序步骤。完成每一步的状态描述。保证终止所有有效输入。

执行模型 执行不需要纯粹是顺序的：现代算法可以是顺序的、并行的、中断驱动的或硬件加速的。

输入 → 过程 → 输出

历史注释：“算法”的词源 (Historical Note: Etymology of “Algorithm”)

算法这个词在历史上与花剌子米有关。该术语反映了符号计算和程序推理的悠久历史。算法植根于使推理系统化的理念。

为什么要学习算法理论？ (Why Study Algorithm Theory?)

仅仅知道问题可以解决是不够的。我们也关心它是如何解决的，是否有效解决。算法理论比较方法，证明正确性，并寻求最佳计算策略。效率是计算科学研究的一部分，而不是事后的想法。

算法的完整描述 (A Complete Description of an Algorithm)

- 做什么 (What)**：明确要解决的问题。
- 怎么做 (How)**：描述算法方法。
- 为什么正确 (Why)**：给出正确性证明。
- 有多快 (How fast)**：进行复杂度分析。

数学归纳法 (Mathematical Induction)

数学归纳法 (Mathematical Induction) 证明了对于所有自然数或无限族索引情况的陈述 $P(n)$ 。当每个案例都依赖于之前的案例时，它特别有用。归纳法是递归定义 (Recursive Definition) 的证明类似物。

归纳证明的标准结构 (Standard Structure of an Induction Proof)

明确声明状态 $P(n)$ 。基本情况验证第一个索引的 $P(n_0)$ 。归纳假设 (Inductive Hypothesis) 假设 $P(k)$ 对于任意 $k \geq n_0$ 成立。归纳步骤 (Inductive Step) 使用假设证明 $P(k+1)$ 。

为什么归纳法是合法的？ (Why Is Induction Legitimate?)

多米诺骨牌直觉 基本情况推倒了第一张多米诺骨牌。归纳步骤确保每张倒下的多米诺骨牌都会导致下一张倒下。因此，该属性在所有情况下传播。 $P(n_0) \wedge (P(k) \Rightarrow P(k+1)) \Rightarrow P(n)$ 对所有 $n \geq n_0$

归纳示例：前 n 个整数的和 (Induction Example: Sum of the First n Integers)

命题 $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ 。

基本情况：当 $n = 1$ 时， $1 = \frac{1 \cdot 2}{2}$ 。

归纳假设： $\sum_{i=1}^k i = \frac{k(k+1)}{2}$ 。

归纳步骤：

$$\begin{aligned}\sum_{i=1}^{k+1} i &= \sum_{i=1}^k i + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) \\ &= \frac{(k+1)(k+2)}{2}.\end{aligned}$$

因此，该公式对于所有 $n \geq 1$ 都成立。

第二个归纳示例和实用说明 (A Second Induction Example and a Practical Note)

命题 $n^3 - n$ 可被 3 整除 对所有 $n \in \mathbb{N}$ 。

基本情况： $n = 1$ 时， $1^3 - 1 = 0$ 。

归纳步骤: 假设 $k^3 - k$ 可被 3 整除, 则 $(k+1)^3 - (k+1) = k^3 - k + 3k(k+1)$,

右侧两项均可被 3 整除。

实用说明 有些作者证明了 $P(k) \Rightarrow P(k+1)$; 其他人写为 $P(k-1) \Rightarrow P(k)$ 。逻辑是相同的。

递归定义 (Recursive Definitions)

递归定义反映了编程中的递归过程。每个递归定义都需要基本情况和递归规则。递归定义最终必须达到基本情况。

示例 $g(1) = 1, g(n) = g(n-1) + n (n \geq 2)$.

递归定义的威力 (The Power of Recursive Definitions)

斐波那契数列 $F_0 = 0, F_1 = 1, F_i = F_{i-1} + F_{i-2} (i \geq 2)$.

当一个项依赖于多个较早的项时, 递归就很强大。递归定义可以是自然且紧凑的。显式的封闭形式通常不太明显。

通过展开解决重复问题 (Solving Recurrences by Unrolling)

示例 $T(n) = 2T(n-1) + 3, T(1) = 1$.

$$\begin{aligned} T(n) &= 2(2T(n-2) + 3) + 3 \\ &= 2^2T(n-2) + 2 \cdot 3 + 3. \end{aligned}$$

展开 k 次后: $T(n) = 2^kT(n-k) + 3 \sum_{i=0}^{k-1} 2^i$.

递推关系何时达到基本情况? (When Does the Recurrence Hit the Base Case?)

令 $n-k=1$, 得到 $k=n-1$ 。代入展开式: $T(n) = 2^{n-1}T(1) + 3 \sum_{i=0}^{n-2} 2^i$.

利用 $T(1) = 1$ 和 $\sum_{i=0}^{n-2} 2^i = 2^{n-1} - 1$, 得到 $T(n) = 2^{n+1} - 3$.

第二种方法: 猜测, 然后证明 (A Second Method: Guess, Then Prove)

递推关系 $f(0) = 2, f(1) = 3, f(n+1) = 3f(n) - 2f(n-1)$.

合理的主张 $f(n) = 2^n + 1$.

计算早期值, 猜测封闭形式, 然后通过归纳法证明。该证明检查两个基本情况并在归纳步骤中使用递归。

为什么有时会出现强归纳法 (Why Strong Induction Sometimes Appears)

有些重复取决于多个早期值。归纳步骤可能需要访问多个先前案例。这激发了多个基本情况和更强的归纳假设。

关键想法 证明结构应该与递归的依赖结构相匹配。

矛盾证明 (Proof by Contradiction)

为了证明 P , 先假设 $\neg P$ 。若该假设导致矛盾, 则 $\neg P$ 为假, 因此 P 为真。 $\neg P \Rightarrow \perp \Rightarrow P$

有力但往往不如直接证明 (Direct Proof) 那么有建设性。

示例: 不存在最大的偶数 (Example: There Is No Largest Even Integer)

假设存在一个最大的偶数 k 。那么对于某个整数 $n, k = 2n$ 。考虑 $k+2 = 2(n+1)$, 这也是偶数。由于 $k+2 > k$, 这与假设相矛盾。

结论 不存在最大偶数。

$\sqrt{2}$ 是无理数 ($\sqrt{2}$ Is Irrational)

假设 $\sqrt{2}$ 是有理数, 并写成最简分数 $\sqrt{2} = \frac{a}{b}$ 。

两边平方得到 $a^2 = 2b^2$, 因此 a 为偶数。令 $a = 2k$, 代入可得 $4k^2 = 2b^2$,

所以 b 也是偶数。这与 $\frac{a}{b}$ 已经是最简分数矛盾, 因此 $\sqrt{2}$ 是无理数。

直接证明、归纳和矛盾 (Direct Proof, Induction, and Contradiction)

风格 核心思想 直接证明 从假设直接得出结论。归纳 通过有限的传播模式证明无限多个索引语句。矛盾 假设相反并得出不可能性。

不同的问题自然意味着不同的证明风格。矛盾往往很强烈, 但建设性较差。

主要算法范式 (Major Algorithmic Paradigms)

贪心算法 (Greedy Algorithm) 分治法 动态规划 (Dynamic Programming) 网络流 随机算法 难处理性 应对难处理性

连接 本讲座中的证明技术支持这些范式的严格开发和分析。

算法发挥作用的地方 (Where Algorithms Matter)

数据库和信息检索 网络和分布式系统 调度和资源分配 数据分析和信号处理 图形和可视化 科学计算和模拟 运筹学和优化 人工智能 计算生物学

结束语 算法设计不仅是为了让事情顺利进行；更是为了让事情顺利进行。这是为了使它们正确、易于理解和高效。

第2周 稳定匹配与五个代表性问题（Stable Matching and Five Representative Problems）

1.1 第一个问题：稳定匹配（1.1 A First Problem: Stable Matching）

核心问题：本地激励措施能否与全局可行性（Feasibility）保持一致？输入：对可接受的合作伙伴的双边严格偏好列表。输出目标：稳定的完美匹配（无阻塞对（Blocking Pair））。方法预览：Gale-Shapley 在 $O(n^2)$ 时间内提出并拒绝。

动机：将住院医师与医院相匹配（Motivation: Matching Residents to Hospitals）

参与者对偏好进行了排名。我们想要一个自我执行的结果。阻塞对（不稳定对）：申请人 x 和医院 y 满足 $1\ x$ 更喜欢 y 而不是其指定的医院， $2\ y$ 更喜欢 x 而不是至少一个当前居民。稳定分配：没有阻塞对的分配。

稳定的婚姻模式（Stable Marriage Model）

n 个男人和 n 个女人。每个人对对方所有成员进行排名（严格顺序）。输入由两组首选项列表组成。

偏好配置文件示例 (3×3):

男子第一名第二名第三名女子第一名第二名第三名 X A B C A Y X Z Y B A C B X Y Z Z A C B C X Z Y

核心定义（Core Definitions）

完美匹配（Perfect Matching）：每个男人都与一个女人完全匹配，反之亦然。阻塞对（不稳定对）（Blocking Pair）(m, w):

$$w \succ_m \mu(m) \quad \text{且} \quad m \succ_w \mu(w).$$

稳定匹配（Stable Matching）：完美匹配，无阻塞对。

示例：这个匹配稳定吗？（Example: Is this Matching Stable?）

建议搭配：(X, C), (Y, B), (Z, A) 任务：检查是否有任何不匹配的对 (m, w) 形成阻塞对。

结果示例：不稳定（Example Result: Not Stable）

该对 (Xavier, Bertha) 是阻塞对（不稳定对）。泽维尔更喜欢伯莎，而不是他指定的搭档。伯莎更喜欢泽维尔而不是她指定的搭档。因此，匹配不稳定。

示例：稳定匹配（Example: A Stable Matching）

匹配：(X, A), (Y, B), (Z, C) 声明：稳定（不存在阻塞对）。

稳定的解决方案总是存在吗？（Does a Stable Solution Always Exist?）

稳定的婚姻（双方）：稳定的匹配始终存在。稳定的室友（非二分）：稳定的匹配可能不存在。4 人反例：每个完美配对都会产生一个阻塞配对。

甲乙

光盘

Gale-Shapley 算法（男士求婚）（Gale-Shapley Algorithm (Men Propose)）

算法草图

- 将所有人初始化为自由人。
- 虽然有些自由男人并没有向每个女人求婚：
m 向他最喜欢的尚未求婚的女人求婚。如果 w 空闲，则参与 (m, w)。否则 w 将 m 与当前未婚夫 m' 进行比较：如果 w 更喜欢 m ，她就会切换到 m ； m' 变得自由。否则 w 拒绝 m 。

终止（受提案约束）（Termination (Bound on Proposals)）

每个人都按照优先顺序递减的顺序求婚。每个提案都针对一位新女性（不重复）。提案总数最多为 n^2 。因此，算法在 $O(n^2)$ 次迭代中终止。

完美（无人能敌）（Perfection (No One Left Unmatched)）

假设某个人在终止时是无与伦比的。那么一定也有一些女人是无与伦比的。如果一个女人无与伦比，就没有人向她求婚。但一个无与伦比的男人在终止妊娠时必须向所有女性求婚。矛盾。因此输出是完美匹配的。

稳定性证明（关键思想）（Stability Proof (Key Idea)）

假设输出中存在阻塞对 (m, w)。如果 m 从未向 w 求婚，那么 m 比 w 更喜欢 $\mu(m)$ ，因此不会阻塞。如果 m 确实向 w 求婚，她拒绝了他，然后才向上交易，所以 w 更喜欢 $\mu(w)$ 而不是 m 。这两种情况都与阻塞相矛盾。因此输出稳定。

稳定匹配：我们拥有什么（Stable Matching: What We Have）

问题：从偏好中找到稳定的匹配。结果：Gale-Shapley 始终返回稳定的匹配。下一个问题：我们如何有效地实施这一点？如果存在多个稳定匹配，则返回哪一个？

$O(n^2)$ 时间的实现 (Implementation for $O(n^2)$ Time)

维护：目前订婚的妻子[m]、丈夫[w] (如果有空，则为 0)。count[m]：man m 提出的提案数量。当前自由人的队列/列表。所有男性和女性的偏好列表。

反向偏好列表 (Inverse Preference Lists)

给定女性 w 和男性 m, m'，我们需要快速比较。预先计算 $\text{rank}[w][m]$ ，表示男性 m 在女性 w 的偏好列表中的位置。那么，w 更喜欢 m 而不是 m' 当且仅当 $\text{rank}[w][m] < \text{rank}[w][m']$ 。

这使得每次比较的复杂度为 $O(1)$ 。

如何建立排名 (How to Build $\text{rank}[w][\cdot]$)

如果女性 w 的偏好数组为 $\text{pref}[1..n]$ ：

$$\text{对 } i = 1, \dots, n: \quad \text{rank}[w][\text{pref}[i]] = i.$$

示例：快速偏好检查 (Example: Fast Preference Check)

假设一个女人 w：

$$\text{rank}[w][3] = 2, \quad \text{rank}[w][6] = 5.$$

那么 w 更喜欢男人 3 而不是男人 6，因为 $2 < 5$ 。

实用要点 (Practical Takeaway)

对于逆数组，每个提议步骤都是 $O(1)$ 。提案数量最多为 n^2 。总运行时间： $O(n^2)$ 。

非唯一性 (Non-Uniqueness)

单个实例可以允许多个稳定匹配。示意图示例：(A, X), (B, Y), (C, Z) 是稳定的；(A, Y), (B, X), (C, Z) 也是稳定的。

稳定解的结构 (Structure of Stable Solutions)

有效伙伴：如果 (m, w) 出现在某些稳定匹配中，则 w 对 m 有效。人最优稳定匹配：每个人都得到他最好的有效伴侣。男性建议 GS 返回男性最佳稳定匹配。返回的匹配对于提议方来说是唯一的。

为什么男性建议 GS 是男性最佳选择 (草图) (Why Men-Proposing GS Is Man-Optimal (Sketch))

假设某个人没有找到他最好的有效伴侣。然后某个男人被一个对他来说有效的女人拒绝了。考虑第一个这样的拒绝事件。使用“女性只向上交易”来在所谓的稳定匹配中派生出阻塞对。矛盾：没有人会被他最好的有效伴侣拒绝。

总结和自然问题 (Summary & a Natural Question)

GS 在 $O(n^2)$ 中计算稳定匹配。男性建议 GS 返回男性最佳稳定匹配。

问：对女性有什么影响？

女性-悲观稳定匹配 (Woman-Pessimal Stable Matching)

向男性求婚的 GS 对每个女性来说都是最糟糕的稳定伴侣。同样，每个女性都会在稳定匹配中得到最差的有效伴侣。证明草图：如果一个女人在其他地方表现更差，那将迫使一个阻挡对。

扩展模型 (实际变体) (Extending the Model (Practical Variants))

常见扩展名：

1. 不可接受的搭档 (禁止比赛)。
2. 集合规模不平等 (一些参与者不匹配)。
3. 有能力的医院 (多对一匹配)。多对一稳定性：(r, h) 阻止，如果两者都认为彼此可以接受，r 更喜欢 h (或不匹配)，并且 h 具有能力或比当前受让人更喜欢 r。

第 1 章 — 简介：一些代表性问题 (Chapter 1 — Introduction: Some Representative Problems)

(二) 五个代表性问题

1.2 五个代表性问题 (1.2 Five Representative Problems)

本节预览五种重复出现的算法原型。我们比较目标设计、结构约束和易处理性。遇到的范式：贪婪、动态规划 (Dynamic Programming) 和基于流的方法。复杂性边界：从多项式时间可解问题到 NP/PSPACE 硬度。

间隔调度 (Interval Scheduling)

输入：具有开始和结束时间的作业。目标：选择不重叠作业的最大子集。约束：所选间隔必须成对兼容。

公元前e时间

间隔调度——兼容性视图 (Interval Scheduling — Compatibility View)

重叠会阻止选择每个间隔。目标是所选工作的数量，而不是总持续时间。

公元前e时间

加权区间调度 (Weighted Interval Scheduling)

每个工作都有一个值（权重）。目标：最大化非重叠作业的总重量。微小的建模变化通常会方法从贪婪规划转变为动态规划。 $w = 4w = 6$
 $w = 7w = 9$

时间

加权调度——权衡 (Weighted Scheduling — Tradeoffs)

更少的间隔可以产生更大的总价值。贪婪计数选择可能不是最优的。

选项 A：许多短期工作，总价值 = 10 选项 B：两项工作，总价值 = 15

加权调度——关键点 (Weighted Scheduling — Key Point)

在非重叠约束下优化权重之和。按完成时间对作业进行排序。令 $p(j)$ 为先前的兼容作业索引。标准递推关系：

$$\text{OPT}(j) = \max \{ \text{OPT}(j-1), w_j + \text{OPT}(p(j)) \}.$$

二分匹配 (Bipartite Matching)

输入：二分图 $G = (L \cup R, E)$ 。目标：最大化端点互不相交的匹配边数量。典型方法是使用增广路径或最大流模型。

独立组 (Independent Set)

输入：图 $G = (V, E)$ 。独立集 (Independent Set) : $S \subseteq V$ ，没有内部边缘。目标：最大化 $|S|$ 。一般图的复杂性：NP 完全。

巴克

德

有竞争力的设施位置 (Competitive Facility Location)

加权图上的两人游戏：玩家交替选择顶点。如果某个顶点与已选定的顶点相邻，则该顶点将被禁止。每个玩家都最大化所选的总重量。复杂性：PSPACE 完整。

7 4 3

2 6 5

五个问题，五个教训 (概述) (Five Problems, Five Lessons (Overview))

问题 典型方法 典型时间复杂度 类 区间调度 (Interval Scheduling) 贪婪 $O(n \log n)$ P 加权区间调度 (Weighted Interval Scheduling) 动态规划 $O(n \log n)$ P 二分匹配 流/增广路径 (Augmenting Path) 多时间 P 独立集 -- NP 完全 竞争性设施选址 (Competitive Facility Location) -- PSPACE 完全

经验教训 (Lessons Learned)

首先认识问题的结构类型。将范式与结构相匹配：贪婪、DP、流程、归约或硬度分析。有些问题可以快速准确地得到解决方案；其他则需要近似法、启发法或重新表述。

算法可以对社会过程进行建模，有时会根据提议者的不同而产生内在的偏见。

第3周 算法分析基础 (Basics of Algorithm Analysis)

五个代表性问题 (Five Representative Problems)

区间调度 (Interval Scheduling) : $n \log n$, 贪心算法; 加权区间调度 (Weighted Interval Scheduling) : $n \log n$, 动态规划 (Dynamic Programming) 算法; 二分匹配: nk , 基于最大流的算法; 独立集 (Independent Set) : NP-complete; 竞争设施位置: PSPACE-complete。

要点：算法问题的范围从可有效解决到可证明难以解决。

我们如何评估算法? (How Do We Evaluate an Algorithm?)

完整性：当存在解决方案时，是否能保证解决方案？最优性：它是否返回最佳解决方案？时间复杂度 (Time Complexity)：运行多长时间？空间复杂度 (Space Complexity)：它使用了多少内存？

本章主要关注运行时间和输入大小的增长。

多项式时间与暴力破解 (Polynomial-Time vs. Brute Force)

许多重要的问题都有一个自然的暴力解决方案。暴力破解通常需要 2^n 时间（或更糟）。在现实环境中，这种增长通常是不可接受的。

多项式时间定义：

$$T(N) \leq cN^d, \quad \text{其中 } c > 0, d > 0 \text{ 为常数.}$$

对上界函数 cN^d 而言，输入规模从 N 增至 $2N$ 时，上界增大为原来的 2^d 倍。因此，多项式增长相对可控。

最坏情况 (Worst-Case) 分析 (Worst-Case Analysis)

最坏情况运行时间：所有大小为 N 的输入的最大运行时间。平均情况 (Average-Case) 运行时间：大小为 N 的随机输入的预期运行时间。

为什么最坏情况通常是首选：通过严格的保证更容易分析。输入分布通常是未知的或不切实际的。针对一种分布调整的算法可能在其他分布上失败。

为什么多项式时间被认为是有效的 (Why Polynomial-Time Is Considered Efficient)

形式化观点：高效意味着最坏情况下的多项式运行时间。多项式时间方法通常很实用。摆脱暴力通常会发现有用的问题结构。

细微差别和注意事项：一些多项式时间算法仍然不切实际。当硬输入很少时，一些指数时间方法很有用。示例：单纯形法、Unix grep、 $O(n^2)$ 中的稳定匹配 (Stable Matching)。

为什么它很重要 (Why It Matters)

假设机器每秒执行 10^6 条高级指令。不同增长率所需时间如下：

增长率	$n = 10^2$	$n = 10^3$	$n = 10^4$	$n = 50$
n	10^{-4} s	10^{-3} s	10^{-2} s	5×10^{-5} s
$n \log_2 n$	6.6×10^{-4} s	0.010 s	0.133 s	2.8×10^{-4} s
n^2	0.01 s	1 s	100 s	2.5×10^{-3} s
n^3	1 s	10^3 s	10^6 s	0.125 s
1.5^n	—	—	—	6.4×10^2 s
2^n	—	—	—	1.13×10^9 s
$n!$	—	—	—	$\gg$ 宇宙年龄

多项式增长尺度：指数/阶乘增长很快就变得不可行。

渐近增长顺序 (Asymptotic Order of Growth)

$$T(n) = O(f(n)) \iff \exists c > 0, n_0 > 0, \text{ 使得对所有 } n \geq n_0, T(n) \leq cf(n).$$

$$T(n) = \Omega(f(n)) \iff \exists c > 0, n_0 > 0, \text{ 使得对所有 } n \geq n_0, T(n) \geq cf(n).$$

$T(n) = \Theta(f(n)) \iff T(n) = O(f(n))$ 且 $T(n) = \Omega(f(n))$. 对于 $T(n) = 32n^2 + 17n + 32$:

正确: $O(n^2), O(n^3), \Omega(n^2), \Omega(n), \Theta(n^2)$. 错误: $O(n), \Omega(n^3), \Theta(n), \Theta(n^3)$.

符号和符号的常见滥用 (Notation and a Common Abuse of Notation)

严格来说, $O(f(n))$ 表示一组函数。更精确的表示法: $T(n) \in O(f(n))$.

幻灯片和文本中的常见速记为 $T(n) = O(f(n))$ 。

警告：类似的陈述

“任何比较排序算法至少需要 $O(n \log n)$ 次比较”

作为下限主张毫无意义。使用 $O(\cdot)$ 表示上界，使用 $\Omega(\cdot)$ 表示下界。

渐近符号的性质 (Properties of Asymptotic Notation)

传递性

如果 $f = O(g)$ 且 $g = O(h)$, 则 $f = O(h)$ 。如果 $f = \Omega(g)$ 且 $g = \Omega(h)$, 则 $f = \Omega(h)$ 。如果 $f = \Theta(g)$ 且 $g = \Theta(h)$, 则 $f = \Theta(h)$ 。

可加性

如果 $f = O(h)$ 且 $g = O(h)$, 则 $f + g = O(h)$ 。如果 $f = \Omega(h)$ 且 $g = \Omega(h)$, 则 $f + g = \Omega(h)$ 。如果 $f = \Theta(h)$ 且 $g = \Theta(h)$, 则 $f + g = \Theta(h)$ 。

一些常见函数的渐近界 (Asymptotic Bounds for Some Common Functions)

具有正首项系数的 d 次多项式是 $\Theta(n^d)$ 。对于某个常数 d ，多项式时间意味着 $O(n^d)$ 。对数底仅相差一个常数因子。对于每个 $x > 0$ ，有 $\log n = O(n^x)$ 。对于每个 $r > 1$ 且 $d > 0$ ，有 $n^d = O(r^n)$ 。

要点：

每个指数最终支配每个多项式。对数的增长速度比每个正多项式幂都慢。

线性时间： $O(n)$ (Linear Time: $O(n)$)

示例：计算 n 个数字的最大值。

伪代码

```
max = A[1]
for i = 2 to n:
    if A[i] > max:
        max = A[i]
return max
```

一次通过输入。运行时间与输入大小成正比。

另一个线性时间示例：合并两个排序列表 (Another Linear-Time Example: Merging Two Sorted Lists)

给定排序列表 $A = (a_1, \dots, a_n)$ 和 $B = (b_1, \dots, b_n)$ ，产生一个排序输出。

伪代码思想：初始化 $i \leftarrow 1, j \leftarrow 1$ ；当 $i \leq n$ 且 $j \leq n$ 时，追加 $A[i]$ 、 $B[j]$ 中较小的元素；最后追加未处理列表的剩余后缀。

每个元素只移动一次。两个大小为 n 的列表的总时间： $O(n)$ 。

$O(n \log n)$ 时间 ($O(n \log n)$ Time)

常见于分治算法中。典型的例子：合并排序、堆排序。

最大空闲间隔示例。输入：到达时间 $x_1, \dots, x_n$ 。目标：无到达的最大时间间隔。方法：对时间进行排序，然后扫描相邻的间隙。

$$\underbrace{O(n \log n)}_{\text{排序}} + \underbrace{O(n)}_{\text{扫描}} = O(n \log n).$$

也称为线性对数时间 (linearithmic time) 。

二次时间： $O(n^2)$ (Quadratic Time: $O(n^2)$)

示例：最近的点对 (暴力)

给定点 $(x_1, y_1), \dots, (x_n, y_n)$ ，测试每一对并保留最小平方距离：

$$\min_{1 \leq i < j \leq n} ((x_i - x_j)^2 + (y_i - y_j)^2).$$

循环草图：初始化 $\text{best} \leftarrow +\infty$ ；对 $i = 1, \dots, n$ 以及 $j = i + 1, \dots, n$ ，用点对 (i, j) 更新最优值。

比较时不需要平方根。尽管暴力算法的 $O(n^2)$ 运行时间看起来难以避免，但分治法可将其改进为 $O(n \log n)$ 。

三次时间： $O(n^3)$ (Cubic Time: $O(n^3)$)

示例：集合不相交。给定集合 $S_1, \dots, S_n \subseteq \{1, 2, \dots, n\}$ ，确定是否存在一对不相交集。迭代所有对 (S_i, S_j) 。对于一组中的每个元素，测试另一组中的成员资格。通过恒定时间隶属度测试，这会产生 $O(n^3)$ 式界限。

注意：如果成员资格测试慢于 $O(1)$ ，则总运行时间可能会更差。

多项式时间： $O(n^k)$ (Polynomial Time: $O(n^k)$)

示例：大小为 k 的独立集给定一个图，是否有 k 个顶点在任意对之间没有边？枚举所有 k 顶点子集。检查每个候选子集是否独立。检查一个子集的成本为 $O(k^2)$ 。

$$\binom{n}{k} \cdot O(k^2) = O(n^k), \quad \text{其中 } k \text{ 为常数.}$$

如果 k 被视为常数，则为多项式时间。当 k 很大时仍然常常不切实际。

指数时间 (Exponential Time)

示例：最大独立集给定一个图，计算最大独立集。枚举所有顶点子集。测试每个子集的独立性。保留迄今为止看到的最大有效子集。

$\#\{\text{由 } n \text{ 个顶点构成的子集}\} = 2^n$ 直接算法的运行时间： $O(n^2 2^n)$ 。

主要教训：穷举搜索很快就变得不可行；渐近分析是必不可少的。

第4周 图论基础 (Graph Fundamentals)

图基础知识和符号 (Graph Basics and Notation)

无向图 (Undirected Graph) 模型: 无向图是一对 $G = (V, E)$, 其中 $n = |V|$, $m = |E|$, V 是顶点集, $E \subseteq \{\{u, v\} : u, v \in V, u \neq v\}$ 是边集。

顶点 (节点) 代表实体; 边代表对称关系。图编码结构: 邻接性、可达性 (Reachability)、层次结构、依赖性。单线动机: 许多系统 (通信、调度、状态空间) 自然地建模为图。

图形表示: 邻接矩阵 (Adjacency Matrix) 与列表 (Graph Representations: Adjacency Matrix vs List)

邻接列表 (Adjacency List): 每个顶点 u 存储其邻居列表 $N(u)$ 。空间为 $O(m + n)$; 查询边 (u, v) 通常需要 $O(\deg(u))$; 枚举全部边需要 $O(m + n)$ 。

邻接矩阵 (Adjacency Matrix): 对于编号为 $1, \dots, n$ 的顶点, 定义 $A \in \{0, 1\}^{n \times n}$, 且 $A_{uv} = 1 \iff \{u, v\} \in E$ 。空间为 $O(n^2)$; 查询边 (u, v) 需要 $O(1)$; 枚举全部边需要 $O(n^2)$ 。

要点: 邻接表 (Adjacency List) 通常在稀疏图 (Sparse Graph) 上是首选 ($m \ll n^2$)。

路径、循环和连通性 (Paths, Cycles, and Connectedness)

路径和简单路径 (Simple Path) 路径是一个顶点序列 $P = (v_1, v_2, \dots, v_k)$

使得对于所有 $i = 1, \dots, k - 1$, 都有 $\{v_i, v_{i+1}\} \in E$ 。如果顶点两两不同, 则称为简单路径 (Simple Path)。

简单环 (Simple Cycle) 是序列

$$(v_1, v_2, \dots, v_k, v_1), \quad k \geq 3,$$

其中 $v_1, \dots, v_k$ 两两不同, 且每个相邻顶点对 (包括 $\{v_k, v_1\}$) 之间都有边。

连通图: 无向图是连通的, 当且仅当对每个 $u, v \in V$, 都存在从 u 到 v 的路径。

树和有根树 (Trees and Rooted Trees)

树 树是一种无向图, 它同时具有: 连接的、非循环的。

有根树: 选择根 $r \in V$ 并将每条边定向为远离 r 。由此得到父子关系、深度、祖先概念, 以及遍历和动态规划使用的层次结构。

基本树定理 (Fundamental Tree Theorem)

定理 如果 T 是一棵有 n 个顶点的树, 则 $|E(T)| = n - 1$ 。

等价直觉 (无向有限图) 以下任何两个都强制第三个:

1. 连接, 2 个非循环, 3 个恰好 $n - 1$ 条边。

样张草图 感应 n 。每棵树都有一片叶子。删除叶子及其关联边: 剩下的图是一棵有 $n - 1$ 个顶点、有 $n - 2$ 条边的树。将叶子添加回去以获得 $n - 1$ 条边。

BFS 设置: 连接性和最短路径 (BFS Setup: Connectivity and Shortest Paths)

核心问题 s-t 连接性: t 是否可以从源 s 到达? s-t 未加权图中的最短路径。

BFS 的分层视图始于 $L_0 = \{s\}$,

然后, 对于 $i \geq 0$, 将 L_{i+1} 定义为与 L_i 中顶点相邻且此前未被发现的顶点集合。

距离解释 每层 L_i 正是距 s 的图距离 i 处的顶点集。因此, BFS 解决了未加权图中的最短路径。

BFS 属性和复杂性 (BFS Properties and Complexity)

BFS 树边的层级性质: 如果 T 是 BFS 树且 $(x, y) \in E$, 则 $|\text{level}(x) - \text{level}(y)| \leq 1$ 。

为什么重要: 边缘不能跳过层, 保留最短路径结构。这控制二分性和距离界限的正确性参数。

时间复杂度 (Time Complexity): 每个顶点处理一次, 每个邻接表扫描一次。对于无向图,

$$\sum_{u \in V} \deg(u) = 2m, \quad \text{因此}$$

$O(m + n)$ 。

连接的组件 (Connected Components)

定义 连通分量 (Connected Component) 是无向图的最大连通子图。

从起始顶点开始的可达集: 对于 $s \in V$, BFS/DFS 从 s 发现的顶点恰好是包含 s 的连通分量。

从每个未访问的顶点运行 BFS/DFS 以枚举所有组件。整个图的总复杂度: $O(m + n)$ 。应用 (简要): 网格邻接处的洪水填充。

二分图 (Bipartite Graph) 和奇循环 (Bipartite Graphs and Odd Cycles)

定义 如果 V 可以分为两个集合 V_0, V_1 , 使得每条边的两个端点分别位于两个集合中, 则图是二分图。

定理 如果图是二部图, 则它没有奇循环。

原因: 在任何环中, 颜色必须按 $0, 1, 0, 1, \dots$ 交替并返回起始颜色, 因此环长必须为偶数。

中心特征 一个图是二分图当且仅当它不包含奇环（Odd Cycle）。

二分性 BFS 测试（BFS Test for Bipartiteness）

BFS 级别的奇偶校验 在每个连接的组件中运行 BFS。将偶数级别分配给一侧，将奇数级别分配给另一侧。

如果没有边连接同一层中的两个顶点，则该图是二分图。如果边 (x, y) 出现在一级 L_j 内，则二分性失败。

奇循环构造 设 z 为 L_i 层 BFS 树中 x, y 的最低公共祖先。组合树路径 $z \rightarrow x, z \rightarrow y$ 和边 (x, y) 产生循环长度 $1 + (j - i) + (j - i) = 2(j - i) + 1$,

这很奇怪。

有向图（Directed Graph）和可达性（Directed Graphs and Reachability）

有向图 有向图是 $G = (V, E)$ ，边为有序对 (u, v) 。方向是结构的一部分。

可达性 BFS/DFS 仍然适用于仅遵循传出边缘。一般来说，可达性不再是对称的。

强连通性（Strong Connectivity） 如果 $u \rightsquigarrow v$ 且 $v \rightsquigarrow u$ ，则顶点 u, v 相互可达。如果每对相互可达，则该图是强连接的。

强连接特性（Strong Connectivity Characterization）

引理 固定任意顶点 $s \in V$ 。有向图是强连通的，当且仅当：

- 从 s 可以到达每个顶点；
- 每个顶点都可以到达 s 。

算法（线性时间） 1 选择任意 s 。

- 从 G 中的 s 运行 BFS。
- 构建反向图 G^{rev} 。
- 从 G^{rev} 中的 s 运行 BFS。
- 当且仅当两个搜索都访问所有顶点时返回 true。复杂度： $O(m + n)$ 。

为什么反转边有效（Why Reversing Edges Works）

直觉 边反转将 G 中的“可以到达 s ”转换为 G^{rev} 中的“从 s 可以到达”。

第一次 BFS 检查对所有 v 是否都有 $s \rightsquigarrow v$ ；在 G^{rev} 中进行的第二次 BFS 检查原图中是否对所有 v 都有 $v \rightsquigarrow s$ 。二者共同证明 s 与所有顶点相互可达，从而由传递性得到强连通性。

DAG 和拓扑排序（DAGs and Topological Ordering）

DAG定义：有向无环图（Directed Acyclic Graph, DAG）。拓扑排序（Topological Ordering）是顶点序列 $v_1, v_2, \dots, v_n$ ，满足对每条边 (v_i, v_j) 都有 $i < j$ 。

解释：所有优先约束都指向前方。

为什么拓扑顺序排除有向环：如果存在环，则顺序将意味着 $i_1 < i_2 < \dots < i_k < i_1$ ，这是不可能的。

源引理和拓扑序的存在性（Source Lemma and Existence of Topological Order）

引理 每个 DAG 都有一个源（入度（Indegree）为 0 的顶点）。

证明想法 假设每个顶点的入度至少为 1。重复跟踪传入的边；在有限图中，某些顶点重复，产生有向循环——矛盾。

定理 每个 DAG 都有一个拓扑顺序。

归纳构造 选择一个源，首先放置它，删除及其出边，然后在剩余的 DAG 上递归。

线性时间拓扑排序（Linear-Time Topological Sorting）

算法（卡恩方法） 1 计算入度计数 $\text{count}[w]$ 。

- 用所有入度为 0 的顶点初始化队列/集合 S 。

```
while S 不为空:
    从 S 中移除一个顶点 v，并输出 v
    对每条出边 (v, w):
        count[w] <- count[w] - 1
        如果 count[w] = 0，则将 w 插入 S
```

复杂性 每个顶点最多进入/离开 S 一次；每条边都被处理一次： $O(m + n)$ 。

DAG 与拓扑序的等价性（DAG $\Leftrightarrow$ Topological Order）

定理 对于有向图 G ，以下等价：

- G 是一个 DAG。
- G 具有拓扑排序。

样张草图 ($2 \Rightarrow 1$): 拓扑顺序禁止通过索引矛盾进行有向循环。($1 \Rightarrow 2$): 源引理 + 重复源移除构造一个顺序。

章节摘要 (修订表) (Chapter Summary (Revision Sheet))

模型: $G = (V, E)$, 其中 $n = |V|$, $m = |E|$ 。邻接矩阵占用 $O(n^2)$ 空间, 邻接列表占用 $O(m + n)$ 空间, 后者更适合稀疏图。树是具有 $n - 1$ 条边的连通无环图。BFS 层给出精确的无权距离, 并以 $O(m + n)$ 时间运行; BFS/DFS 也可在线性时间内求出连通分量。图为二分图当且仅当它不含奇环。有向图的强连通性可通过分别在 G 和 G^{rev} 中运行 BFS 来测试。DAG 存在拓扑序, Kahn 算法的运行时间为 $O(m + n)$ 。

第5-6周 贪心算法 (一) (Greedy Algorithms - Part 1)

什么是贪心算法 (Greedy Algorithm) ? (What Is a Greedy Algorithm?)

贪心算法逐步构建解决方案。在每一步中, 它都会选择当时看起来最好的选项。贪心规则很容易表述。困难的部分是证明贪心规则会导致最优的全局解决方案。

关键信息 局部选择并不自动意味着全局最优。正确性必须被证明, 而不是猜测。

间隔调度 (Interval Scheduling)

我们被分配了 n 个工作。作业 j 在时间 s_j 开始并在时间 f_j 结束。如果两个工作不重叠, 则它们是兼容的。目标: 找到相互兼容的作业的最大基数集。

定义 (符号) 作业 $j = [s_j, f_j)$

$$\text{Compatible}(i, j) \iff f_i \leq s_j \text{ 或 } f_j \leq s_i.$$

最大 $|A|$ 前提是 A 中的所有作业都是成对兼容的。

候选贪婪规则 (Candidate Greedy Rules)

按某种顺序考虑作业, 如果每个作业与已选择的作业兼容, 则接受该作业。

1. 最早开始时间: 按 s_j 排序
2. 最早结束时间: 按 f_j 排序
3. 最短间隔: 按 $f_j - s_j$ 排序
4. 最少冲突: 按 c_j 排序

其中 c_j 是与作业 j 冲突的作业数量。

观察 有几条规则看起来似乎是合理的。其中只有一个是正确的。

不良贪婪规则的反例 (Counterexamples to Bad Greedy Rules)

最早开始时间: 失败 非常早的间隔可能会很长并阻塞许多短的兼容间隔。

最短间隔: 失败 非常短的间隔可能在本地看起来很有吸引力, 但会破坏更大的兼容计划。

冲突最少: 失败 冲突很少并不能保证全局最佳放置。

正确规则: 最早完成时间优先 (Correct Rule: Earliest Finish Time First)

贪心法则 按完成时间递增的顺序考虑作业。接受与已选择的工作兼容的每项工作。 $f_1 \leq f_2 \leq \dots \leq f_n$

- 设 j 为最后选择的作业。那么工作 j 是兼容的如果 $s_j \geq f_{j^*}$,

其中 j^* 是最后一个已被选中的作业。

算法 初始化 $A \leftarrow \emptyset$ 。对 $j = 1, \dots, n$:

如果作业 j 与 A 兼容, 则设置 $A \leftarrow A \cup j$ 。返回 A 。运行时间: $O(n \log n)$ 。

为什么最早完成时间是最佳时间 (Why Earliest Finish Time Is Optimal)

设 $i_1, i_2, \dots, i_k$ 为贪心算法选择的作业, 令 $j_1, j_2, \dots, j_m$ 为最优解中的作业。交换结构: 假设两个解尽可能长地保持一致, 即 $i_1 = j_1, \dots, i_r = j_r$ 。在第一个不同的位置, 贪心解选择 i_{r+1} , 最优解选择 j_{r+1} 。

因为贪心算法选择了完成时间最早的兼容作业, 所以 $f_{i_{r+1}} \leq f_{j_{r+1}}$ 。用 i_{r+1} 替换 j_{r+1} 后, 解仍然可行且所选作业数不减少, 这与 r 的最大性矛盾。

定理 贪婪是最优的。

区间划分 (Interval Partitioning)

我们有 n 个讲座。讲座 j 在时间 s_j 开始并在时间 f_j 结束。所有讲座都必须安排。目标: 使用最少数量的教室, 以便不会将两个重叠的讲座分配到同一个房间。教室的最小数量受到重叠讲座必须放置在不同房间的限制。对比间隔调度选择最大的兼容子集。间隔分区以最少的资源调度所有间隔。

基本下界 (A Fundamental Lower Bound)

定义 (深度) 一组间隔的深度是包含任何单个时间点的间隔的最大数量:

$$\text{depth} = \max_t |\{j : t \in [s_j, f_j)\}|.$$

所需教室数量下限 $\geq$ 深度。

如果 d 个间隔在时间 t 重叠, 则这 d 个间隔必须放置在 d 个不同的教室中。因此, 任何有效的时间表都必须至少使用深度教室。

区间划分 (Interval Partitioning) 的贪心规则 (Greedy Rule for Interval Partitioning)

贪心法则 按开始时间递增的顺序处理讲座；将每个讲座分配到任何兼容的教室；如果不存在，则开设一个新教室。 $s_1 \leq s_2 \leq \dots \leq s_n$ 。

对于每个教室，保留分配给它的最后一堂课的结束时间。使用按当前完成时间排序的优先级队列。

伪代码 初始化 $d \leftarrow 0$ 。对 $j = 1, \dots, n$ ：如果讲座 j 与某个教室 k 兼容，则将其分配给 k ；否则分配教室 $d + 1$ ，分配讲座 j ，并设置 $d \leftarrow d + 1$ 。

运行时间： $O(n \log n)$ 。

为什么分区贪心规则是最优的 (Why the Partitioning Greedy Rule Is Optimal)

令 d = 贪婪使用的教室数量。考虑贪心算法为某个讲座 j 打开教室 d 的时刻。

讲座 j 与所有先前使用的 $d - 1$ 教室不兼容。所以已经有 $d - 1$ 个讲座与 j 重叠。因此，在时间 $s_j + \epsilon$ 处，有 d 个重叠的讲座。

因此 深度 $\geq d$ 。每个时间表都满足 $\text{OPT} \geq \text{深度}$ ，所以 $\text{OPT} \geq \text{深度} \geq d$ 。由于贪心算法恰好使用 d 个教室，故 $d = \text{OPT}$ 。

最小化最大延迟 (Minimizing Maximum Lateness)

一台机器一次处理一项作业。每个作业 j 都有处理时间 t_j 和截止时间 d_j 。如果作业 j 在时间 s_j 开始，则其完成时间为 $f_j = s_j + t_j$ 。

定义（迟到） $\lambda_j = f_j - d_j$ 。

最大迟到时间为 $L = \max_j \lambda_j$ 。

最小化 L 。

尽量减少迟到的候选人规则 (Candidate Rules for Minimizing Lateness)

可能的贪心规则：1、处理时间最短优先：按 t_j 2 排序 从最早的截止日期开始：按 d_j 3 排序，首先是最小的松弛：按 $d_j - t_j$ 排序

关键信息：其中一些规则是错误的。我们需要一个能够正确最小化 $L = \max_j \lambda_j$ 的模型。

两条不好的规则 (Two Bad Rules)

规则 1：处理时间最短优先 — 失败示例： $t_1 = 1, d_1 = 100, t_2 = 10, d_2 = 10$ 。

首先完成较短的工作会不必要地拖延紧急工作。

规则 2：最小松弛优先 — 失败示例： $t_1 = 10, d_1 = 10, t_2 = 1, d_2 = 2$ 。

松弛值： $d_1 - t_1 = 0, \quad d_2 - t_2 = 1$ 。

所以最小的松弛首先会选择作业 1，这并不是最优的。

结论 最短处理时间优先是不正确的。最小松弛优先是不正确的。

正确规则：最早截止日期优先 (Correct Rule: Earliest Deadline First)

贪心法则 按截止日期递增的顺序安排作业。

重命名作业，以便 $d_1 \leq d_2 \leq \dots \leq d_n$ 。

算法

初始化 $t \leftarrow 0$ 。对 $j = 1, \dots, n$ ：

将作业 j 分配给区间 $[t, t + t_j]$ ；设 $s_j \leftarrow t$ 且 $f_j \leftarrow t + t_j$ ；更新 $t \leftarrow t + t_j$ 。

输出：开始时间 s_j 和结束时间 f_j 。

无空闲时间和反转 (No Idle Time and Inversions)

观察 存在一个没有空闲时间的最佳时间表。

可以消除空闲时间，而不会使任何工作相对于其他工作晚于完成。

观察 贪心调度没有反转。

定义（反转）：假设作业已按截止日期编号，使得 $d_1 \leq d_2 \leq \dots \leq d_n$ 。若 $i < j$ 但作业 j 被安排在作业 i 之前，则称 (i, j) 为一个反转。

关键信息 在最佳截止日期排序计划中，较早的截止日期不应置于较晚的截止日期之后。

交换相邻反转 (Swapping an Adjacent Inversion)

命题 交换两个连续的反转作业会将反转数量减少 1，并且不会增加最大延迟 (Maximum Lateness)。

设相邻反转中作业 j 位于作业 i 之前，其中 $i < j$ ，因而 $d_i \leq d_j$ 。交换后，其他作业的完成时间不变；作业 i 提前完成，所以 $\lambda'_i \leq \lambda_i$ 。

作业 j 交换后的完成时间等于交换前作业 i 的完成时间 f_i ，因此

$$\lambda'_j = f_i - d_j \leq f_i - d_i = \lambda_i。$$

因此，最大迟到时间不会增加。

为什么最早截止日期优先是最佳选择 (Why Earliest Deadline First Is Optimal)

令 S 为具有最少反转次数的最佳调度。我们可以假设 S 没有空闲时间。

如果 S^* 没有反转，那么它是按非递减期限排序的，因此它正是贪心调度。如果 S^* 有一个反转，那么它有一个相邻的反转。根据前面的说法，交换相邻反转不会增加最大延迟，并且会严格减少反转的数量。

这与 S^* 的定义相矛盾。**定理** 最早的截止日期优先是最佳的。

标准贪婪证明技术 (Standard Greedy Proof Techniques)

- 贪婪保持领先 证明每一步之后，贪婪解决方案至少与其他解决方案一样好。
示例：间隔调度。
- 结构下界 (Lower Bound) 证明每个可行解必须满足一个下界。
显示贪婪达到了该界限。示例：区间划分。
- 交换论证 (Exchange Argument) 从最优解开始，将其转化为贪婪解，且不使其变得更糟。示例：尽量减少迟到。

外卖 正确的贪婪算法通常依赖于这些证明模式之一。

最佳离线缓存 (Optimal Offline Caching)

我们得到：容量 (Capacity) 为 k 的缓存，请求序列为 $d_1, d_2, \dots, d_M$ 。定义：缓存命中：请求的项已在缓存中。缓存未命中：请求的项不在缓存中。如果未命中，则必须将所请求的项目放入缓存中，如果缓存已满，则必须逐出一个缓存的项目。

缓存未命中的最小次数

一个小例子 (A Small Example)

令 $k = 2$ ，初始缓存 = {a, b}，请求序列 = a, b, c, b, c, a, a, b。

事实 最佳逐出计划恰好会产生 2 次缓存未命中。

步骤丢失后的步骤请求缓存？

- a {a, b} 否 2 b {a, b} 否 3 c {b, c} 是 4 b {b, c} 否 5 c {b, c} 否 6 a {a, b} 是 7 a {a, b} 否 8 b {a, b} 否

贝拉迪规则 (Belady's Rule)

贪心法则 当缓存未命中时，逐出下一个请求最远发生的项目。

将此策略称为“最遥远的未来”(FF)。**定理** FF 是最佳的离线驱逐 (Eviction) 计划。

直觉 如果某件物品在很长一段时间内都不需要，那么保留它是最不紧急的。

减少驱逐时间表 (Reduced Eviction Schedules)

定义 (简化计划) 简化计划仅在请求该项目时将项目插入到缓存中。

命题 任何调度都可以转换为缩减的调度，不再有缓存未命中。

在项目真正被请求之前提前将它装入缓存不会带来额外收益，因此可把插入推迟到请求发生时，而不增加缓存未命中次数。因此，我们可以假设存在一个最优的简化调度。

结论 仅将 FF 与最佳缩减时间表进行比较就足够了。

最佳缓存的归纳不变量 (Inductive Invariant for Optimal Caching)

我们通过对已处理请求数量的归纳来证明 FF 的最优性。**不变** 存在一个最优简化调度 S ，它通过前 j 个请求与 FF 一致。

现在考虑请求 d_{j+1} 。简单案例：

- d_{j+1} 已在缓存中。
- S 和 FF 驱逐相同的项目。
唯一重要的情况是：FF 驱逐 e , S 驱逐 f , $e \neq f$ 。

交换步骤 (The Exchange Step)

通过在请求 d_{j+1} 时逐出 e 而不是 f ，从 S 构建新的调度 S' 。现在 S' 又同意 FF 一步。让

$j' = S$ 和 S' 不同的第一个未来时间，

设在时间 j' 时请求的项目为 g 。

- $g = e$ ：不可能，因为 FF 选择了 e 作为未来使用最远的项。
- $g = f$ ：那么 S' 并不更糟，因为它将 f 保留在缓存中。
- $g \neq e, f$ ：调整后面的驱逐，使 S 和 S' 再次对齐。

结论 S' 仍然是最优的，并且与 FF 的一致性多了一步。通过归纳法，

FF 是最佳的。

线下和线上视角 (Offline and Online Perspectives)

离线两种设置：提前知道整个请求序列。在线：未来的请求未知。

策略：FF (最佳离线)、LRU (最近最少使用)、LIFO (后进先出)。事实：FF 是离线最优的，LRU 是 k 竞争的，而 LIFO 可能是任意糟糕的。

解释 FF 可以被视为 LRU 的未来感知类似物。

有向图 (Directed Graph) 中的最短路径 (Shortest Paths in a Directed Graph)

最短路径实例包括：a directed graph $G = (V, E)$

给定源节点 s 、目标节点 t ，以及每条边 $e \in E$ 的非负长度 $\ell_e \geq 0$ 。

目标：找到一条从 s 到 t 且总长度最小的有向路径。如果 $P = (e_1, e_2, \dots, e_r)$,

则 $\ell(P) = \sum_{q=1}^r \ell_{e_q}$.

Dijkstra 算法 (Dijkstra's Algorithm)

维护已探索节点集合 S 。对于每个 $u \in S$ ，从 s 到 u 的最短路径距离已经确定。

初始化 $S = \{s\}$ 和 $d(s) = 0$ 。对于每个未探索的节点 v ，定义

$$\pi(v) = \min_{\substack{e=(u,v) \in E \\ u \in S}} (d(u) + \ell_e).$$

贪心步骤：

选择具有最小 $\pi(v)$ 的未探索节点 v ，将 v 加入 S ，并令 $d(v) = \pi(v)$ 。

Dijkstra 算法的正确性 (Correctness of Dijkstra's Algorithm)

不变式 对于每个节点 $u \in S$ ， $d(u)$ 是从 s 到 u 的最短路径长度。

证明是通过 $|S|$ 上的归纳得出的。令 v 为下一个选择的节点。考虑从 s 到 v 的任何路径 P 。令 $x \rightarrow y$ 为 P 离开 S 的第一条边，令 P' 为 P 从 s 到 x 的前缀。然后 $\ell(P) \geq \ell(P') + \ell(x, y) \geq d(x) + \ell(x, y) \geq \pi(y) \geq \pi(v)$ 。

说明： $\ell(P') \geq d(x)$ 由归纳假设可得。

$$d(x) + \ell(x, y) \geq \pi(y) \quad \text{由 } \pi(y) \text{ 的定义可得。}$$

因为 Dijkstra 选择最小的边界值，所以 $\pi(y) \geq \pi(v)$ 。因此令 $d(v) = \pi(v)$ 是正确的。

优先级队列实现 (Priority Queue Implementation)

维护每个未探索的节点 v ：

$$\pi(v) = \min_{\substack{e=(u,v) \in E \\ u \in S}} (d(u) + \ell_e).$$

每一步：提取具有最小 $\pi(v)$ 的未探索节点。当节点 v 添加到 S 时，对于每个出边 $e = (v, w)$,

更新 $\pi(w) = \min\{\pi(w), \pi(v) + \ell_e\}$ 。

实现注意事项 使用以 $\pi(v)$ 作为键控的优先级队列。

总结 (Summary)

主要结果：

- 间隔调度：按照最早完成时间选择作业。
- 间隔分区：按最早开始时间进行处理，贪婪地重用房间。
- 尽量减少迟到：首先按最早的截止日期安排。
- 最佳离线缓存：逐出未来最远请求的项目。
- 最短路径：Dijkstra 通过最小边界距离来增长探索集。

最终消息贪心算法非常强大，因为它们结合了简单的局部规则、高效的实现和强大的正确性证明。

第7周 贪心算法 (二)：最小生成树与聚类 (Greedy Algorithms - Part 2: MST and Clustering)

4.5 最小生成树 (4.5 Minimum Spanning Tree)

最小生成树 (Minimum Spanning Tree, MST) 问题是一个经典的图优化问题。从历史上看，最早的算法之一可以追溯到 1926 年。本节研究问题定义、结构属性、贪婪算法和实现。

最小生成树：形式定义 (Minimum Spanning Tree: Formal Definition)

定义 连通加权图为 $G = (V, E)$ ，其中每条边 $e \in E$ 都有一个实值成本 c_e 。

生成树是一个子集 $T \subseteq E$ ，使得 (V, T) 是一棵跨越所有顶点的树。

等价：以无环且总成本最小的方式连接所有顶点。

目标 最小生成树满足

$$\min_{T \subseteq E} \sum_{e \in T} c_e \quad \text{使得 } (V, T) \text{ 是生成树。}$$

备注 n 个顶点上的生成树恰好包含 n - 1 条边。

为什么最小生成树很重要 (Why Minimum Spanning Trees Matter)

网络设计

• 电话网络；电力网络；液压系统 • 有线电视网络；计算机网络；道路网络

近似算法

• 旅行推销员问题 • 斯坦纳树问题

MST 既是直接优化目标，又是许多算法领域的结构子程序。

间接应用

• 最大瓶颈路径；用于纠错的 LDPC 码；图像配准 • 人脸验证；蛋白质测序数据存储减少 • 湍流流体流动局部建模；无环路以太网桥接

数据分析

• 聚类

MST 既是直接优化目标，又是许多算法领域的结构子程序。

MST 的三种贪心算法 (Three Greedy Algorithms for MST)

克鲁斯卡尔算法 $T = \emptyset$

按权重非递减顺序处理边缘。将边 e 添加到 T，除非这样做会产生循环。反向删除算法 $T = E$

按权重非递增顺序处理边缘。从 T 中删除边 e，除非删除它会断开图的连接。关键事实 这两个规则都是贪婪的边排序策略。

Prim 算法从根顶点 s 开始。向外长一棵树。在每一步中，添加当前树中已有一个端点的最便宜的边。关键事实 所有三种算法都会生成最小生成树。

两个基本结构特性 (Two Fundamental Structural Properties)

暂时假设所有边的成本 c_e 两两不同。

定理 1 (循环性质, Cycle Property) 设 C 为 G 中的任意环， f 为 C 中成本最大的边，则 $f \notin T^*$ 。

定理 2 (割性质, Cut Property) 令 $S \subseteq V$ ，并令 e 为割集 $D(S)$ 中成本最小的边，则 $e \in T^*$ 。

这两条语句是贪婪 MST 算法背后的核心正确性原则。

循环、割断和割集 (Cycles, Cuts, and Cutsets)

定义 1: 循环 循环是形成闭合链的一组边: $a - b, b - c, c - d, \dots, y - z, z - a$ 。

定义 2: 割 割是顶点的任意子集: $S \subseteq V$ 。

定义 3: 割集 (Cutset) 与 S 对应的割集是在 S 中只有一个端点的边的集合:

$$D(S) = \{(u, v) \in E : (u \in S, v \notin S) \text{ 或 } (u \notin S, v \in S)\}.$$

割集恰好包含从 S 到 $V \setminus S$ 的边。

关键引理：循环切割交集 (A Key Lemma: Cycle-Cut Intersection)

命题 1: 对于每个环 C 和每个割集 D ， $|C \cap D|$ 是偶数。

- 循环每次越过切口时就进入集合 S。
- 要回到起点，它也必须离开 S。
- 因此，交叉是成对发生的。
- 因此交叉边的数量是偶数。

校样草图 每个进入 S 的条目都必须与稍后的退出相匹配。因此交集基数是偶数。

切割属性证明 (Proof of the Cut Property)

定理 3: 设 $S \subseteq V$ ，并设 e 为恰有一个端点位于 S 中的最小成本边，则 $e \in T^*$ 。

证明。

- 反设 $e \notin T^*$ 。
- 由于 T^* 是生成树，添加 e 会产生唯一的环 C 。
- 边 e 位于环 C 中。
- 设 $D = D(S)$ 。由于 e 穿过割 S ，所以 $e \in D$ 。
- 根据环与割集交集引理， $C \cap D$ 包含偶数条边。
- 因此存在另一条边 $f \in C \cap D$ ，且 $f \neq e$ 。
- 构造 $T' = T^* \cup \{e\} \setminus \{f\}$ 。
- 那么 T' 仍然是一棵生成树。
- 由于 e 是穿过切口的最小成本边， $c_e < c_f$ 。
- 因此
于是 $\text{cost}(T') < \text{cost}(T^*)$ ，与 T^* 的最小性矛盾。

结论: $e \in T^*$ 。

任何切割中最便宜的边缘总是安全的。

循环属性证明 (Proof of the Cycle Property)

定理 4: 设 C 为任意环, f 为 C 上成本最大的边, 则 $f \notin T^*$.

证明。

1. 反设 $f \in T^*$ 。
2. 从 T^* 中删除 f 。由于 T^* 是一棵树, 删除后会断开并定义一个割 S 。
3. 令 $D = D(S)$, 则 $f \in D$ 。
4. 由于 $f \in C \cap D$, 且环与割集相交于偶数条边, 因此存在另一条边 $e \in C \cap D$, 且 $e \neq f$ 。
5. 构造 $T' = T^* \cup \{e\} \setminus \{f\}$ 。
6. 则 T' 也是生成树。
7. 因为 f 是循环中的最大成本边, $c_e < c_f$ 。
8. 因此
于是 $\text{cost}(T') < \text{cost}(T^*)$, 矛盾。

结论 $f \notin T^*$ 。

因此, MST 中不包含环上成本最大的边。

Prim 算法: 正确性思想 (Prim's Algorithm: Correctness Idea)

Prim 的算法与 Jarník (1930)、Dijkstra (1957) 和 Prim (1959) 相关。

1. 从任意顶点 s 开始。
2. 设 S 为已探索顶点的集合。
3. 在每次迭代中, 考虑 S 中只有一个端点的所有边。
4. 选择成本最小的边。
5. 将该边和新端点添加到正在生长的树中。
通过切割属性, 每个选定的边都是安全的。

每一步, 当前树都可以扩展为 MST。Prim 是正确的, 因为每个贪婪的选择都由割断属性证明是合理的。

Prim 算法: 伪代码 (Prim's Algorithm: Pseudocode)

算法 初级 (G, c) : 对于 V 中的每个 v : $a[v] \leftarrow \infty$

选择任意起始顶点 s $a[s] \leftarrow 0$

为 V 中的每个 v 初始化空优先级队列 Q : 使用键 $a[v]$ 将 v 插入 Q 中

$S \leftarrow \emptyset$

当 Q 不为空时: $u \leftarrow \text{extract-min}(Q)$ $S \leftarrow S \cup \{u\}$

对于与 u 相关的每条边 $e = (u, v)$: 如果 v 不在 S 中且 $c_e < a[v]$: $a[v] \leftarrow c_e$ 减少键($Q, v, a[v]$)

Prim 的算法: 运行时 (Prim's Algorithm: Runtime)

$a[v]$ = 连接 v 到 S 的边的最小成本。

运行时

数组实现: $O(n^2)$, 二叉堆实现: $O(m \log n)$ 。

优先级队列起着与 Dijkstra 算法相同的结构作用, 但具有附着成本而不是最短路径估计。

Kruskal 算法: 正确性思路 (Kruskal's Algorithm: Correctness Idea)

算法思路

• 按权重升序对所有边进行排序。• 将它们——处理。

情况 1 如果添加边 e 形成环, 则丢弃它。原因:

循环属性。

不安全的循环边缘将被立即拒绝。

情况 2 如果添加边 $e = (u, v)$ 不会形成环, 则添加它。原因:

• 令 S 为包含 u 的连通分量 (Connected Component) 的顶点集。• 那么 e 是切割的最小成本边交叉。• 因此 e 通过割断性质是安全的。

所选边的集合始终是某个 MST 的子集。

Kruskal 在拒绝不安全循环边缘和接受安全切割边缘之间交替。

Kruskal 算法: 伪代码 (Kruskal's Algorithm: Pseudocode)

算法 克鲁斯卡尔 (G, C) : 对边进行排序, 以便 $c_1 \leq c_2 \leq \dots \leq c_m$

$T \leftarrow \emptyset$

对于 V 中的每个 u : 制作一个包含 u 的单例集合

对 $i = 1, \dots, m$: 让 $e_i = (u, v)$ 如果 $\text{Find}(u) \neq \text{Find}(v)$: $T \leftarrow T \cup e_i$ 并集(查找(u), 查找(v))

返回T

Kruskal 算法：运行时 (Kruskal's Algorithm: Runtime)

主要数据结构

• 使用并集查找/不相交集并集。• 运算：查找(u)、查找(v)、并集(查找(u)、查找(v))。

运行时：排序需要 $O(m \log n)$ ，并查集操作共需要 $O(m\alpha(m, n))$ 。 $m \leq n^2 \Rightarrow \log m = O(\log n)$ 。

实际上，排序在总运行时间中占主导地位。

删除不同权重假设 (Removing the Distinct-Weights Assumption)

之前的证明假设所有边缘成本都是不同的。• 以任意小的数量扰乱成本，或者 • 按字典顺序打破关系。

Prim 和 Kruskal 仅依赖于边权重的成对比较。比较逻辑less(i, j)：如果成本 (e_i) <成本 (e_j)：返回 true 否则如果成本(e_i) > 成本(e_j)：如果 i < j, 则返回 false：返回 true 否则：返回错误

如果所有成本都是整数，可将边 e_i 的成本替换为

$$c'(e_i) = c(e_i) + \frac{i}{n^2 + 1}, \quad 1 \leq i \leq m \leq n^2.$$

平局打破消除了比较歧义；扰动后选出的 MST 仍然是原始权重下的一棵 MST，但原问题可能有多棵 MST。

4.7 聚类 (4.7 Clustering)

除了网络设计之外，最小生成树也很有用。一个主要的应用是集群。我们现在研究贪婪的 MST 结构如何得出最大间距聚类 (Maximum-Spacing Clustering) 的解决方案。

聚类：问题表述 (Clustering: Problem Formulation)

给定一组 n 个对象 $U = \{p_1, p_2, \dots, p_n\}$,

目标是将对象划分为连贯的组。距离函数 数值 $d(p_i, p_j)$ 测量两个物体的接近程度。对于算法公式，我们研究 U 上边权重为 $d(p_i, p_j)$ 的完整图。基本目标 不同簇中的点应该相距较远。聚类很大程度上取决于所选的距离函数。

应用实例

• 移动自组织网络中的路由 • 基因表达分析 • 文档分类 • 医学图像数据库中的相似性搜索 • 天文物体分类

聚类很大程度上取决于所选的距离函数。

最大间距 k 聚类 (Maximum-Spacing k -Clustering)

定义 1: k 聚类将 U 划分为 k 个非空簇。

假设距离函数满足

$$d(p_i, p_j) = 0 \iff p_i = p_j, \quad d(p_i, p_j) \geq 0, \quad d(p_i, p_j) = d(p_j, p_i).$$

这些条件确保了具有不可辨别的同一性的非负对称距离模型。

定义 2: 间距 聚类的间距是不同聚类中任意一对点之间的最小距离：

$$\text{spacing}(C) = \min_{\substack{p_i \in C_r, p_j \in C_s \\ r \neq s}} d(p_i, p_j).$$

优化问题 给定 k ，找到最大间距的 k 聚类。我们希望簇间分离尽可能大。

k 聚类的贪心算法 (Greedy Algorithm for k -Clustering)

算法

1. 从n个单例集群开始。
2. 反复查找属于不同簇的最接近的一对对象。
3. 合并这两个集群。
4. 当正好剩下 k 个簇时停止。

应用于 U 上的完整加权图，这正是 Kruskal 算法在有 k 个连通分量时提前停止的情况。

单链路聚类 = Kruskal 的早期停止版本。

等效 MST 解释

1. 计算 MST。
2. 删除 $k - 1$ 个最昂贵的边。

因此，MST 结构产生了自然的贪婪聚类规则。

贪心聚类的最优性 (Optimality of Greedy Clustering)

定理。让 $C^* = \{C_1^*, C_2^*, \dots, C_k^*\}$

是通过删除 MST 的 $k - 1$ 条最昂贵边而形成的聚类。那么 C^* 是最大间距的 k 聚类。证明。

1. 令 d^* 为被删除边中成本最小者的成本。
2. 每条剩余的 MST 边的成本至多为 d^* ，因此同一簇中的任意两个顶点由一条边成本均不超过 d^* 的路径连接。
3. 成本为 d^* 的被删除边连接两个不同的 C^* 簇。
4. 因此 C^* 的间距为 d^* 。
5. 考虑任何其他 k 聚类 $C = \{C_1, C_2, \dots, C_k\}$ 。
6. 由于 C 有 k 个簇，而 C^* 也有 k 个簇，若二者不同，则 C^* 的某个簇 C_r^* 包含位于 C 不同簇中的点。
7. 选择这样的点 $p_i, p_j \in C_r^*$ 。沿着从 p_i 到 p_j 的 MST 路径，必有一条边 (p, q) 跨越 C 的两个不同簇。
8. 该路径上的每条边的成本至多为 d^* 。
9. 因此 $d(p, q) \leq d^*$ 。
10. 由于 p 和 q 位于 C 的不同簇中， $\text{spacing}(C) \leq d^*$ 。
11. 因此，任意 k 聚类 C 的间距不大于 d^* 。

结论

C^* 具有最大间距。删除 $k - 1$ 个最大的 MST 边是最优的。

整个故事的摘要 (Summary of the Full Story)

1. MST问题要求生成树的总权值最小。
2. cut 属性标识必须属于 MST 的边。
3. 循环属性标识不能属于MST 的边。
4. Prim 的算法在生长一棵树时重复使用剪切属性。
5. Kruskal 算法在合并组件时重复使用剪切和循环逻辑。
6. 高效的实现在标准设置中使用 $O(m \log n)$ 时间。
- 7.同样的贪心结构解决最大间距聚类。
7. 最大间距 k 聚类是通过删除 MST 的 $k - 1$ 条最大边获得的。

这是由强大的结构定理支持的贪婪范式的典型例子。

讨论问题 (Discussion Questions)

问题 1 为什么 cut 属性证明 Prim 算法的每一步都是合理的？问题2 为什么循环检测在Kruskal算法中至关重要？问题 3 为什么删除 MST 的 $k - 1$ 个最昂贵的边可以使聚类间距最大化？ 这些问题测试证明理解和算法直觉。

第8周 霍夫曼编码与最优前缀码 (Huffman Coding: Optimal Prefix Codes and Greedy Algorithms)

为什么我们需要压缩？ (Why Do We Need Compression?)

给定有限字母表上的文本，我们如何使用位对每个符号进行编码？假设一段文本使用 32 个不同的符号：26个字母，空格字符，几个标点符号。自从 $2^5 = 32$,

固定长度编码可以使用 5 位来表示每个符号。固定长度编码很简单，但它为每个符号分配相同数量的位数。问题 某些符号比其他符号出现的频率更高。固定长度编码忽略了这种频率不平衡。

定长编码 (Fixed-Length Encoding) 浪费信息 (Fixed-Length Encoding Wastes Information)

固定长度编码 压缩直觉 每个符号都有相同的代码长度。 频繁出现的符号应该较短，易于解码。 代码。不利用统计结构。 稀有符号可以接收更长的代码。总编码长度可能会减少。在自然语言中，字符频率非常不均匀：

e、t、a、o、i、n 与 q、z、x。

压缩直觉 为了减少比特总数，请将较短的代码分配给频繁的符号，将较长的代码分配给稀有的符号。

变长编码 (Variable-Length Encoding)

使用可变长度代码，而不是为每个字符分配相同数量的位数。

符号类型 频率 所需代码长度 常见符号 高 短 稀有符号 低 长

如果符号 x 具有相对频率 f_x ，则其对平均码长的贡献为

$$f_x |c(x)|。$$

新难题如果代码具有不同的长度，我们如何知道一个符号在哪里结束以及下一个符号在哪里开始？

可变长度代码中的歧义 (Ambiguity in Variable-Length Codes)

考虑代码 $c(a) = 01, c(b) = 010, c(e) = 1$ 。

位串是什么意思？0101

两种可能的解码是 $0101 = 01|01 = aa$

和 $0101 = 010|1 = be$ 。

原因是a的代码，即01，是b的代码，即010的前缀。关键问题 可变长度代码需要一个保证唯一解码的规则。

前缀代码 (Prefix Codes)

如果对任意两个不同的符号 $x, y \in S$, $c(x)$ 都不是 $c(y)$ 的前缀, 则符号集 S 的编码 c 是前缀码。

示例: $c(a) = 11, c(e) = 01, c(k) = 001, c(l) = 10, c(u) = 000$.

没有一个码字是另一个码字的前缀。因此解码可以明确地进行:

$$1001000001 = 10 \mid 01 \mid 000 \mid 001 \mapsto \text{leuk}.$$

为什么前缀代码很重要 前缀代码可以从左到右解码, 无需分隔符。

作为二叉树的前缀代码 (Prefix Codes as Binary Trees)

二叉树给出了前缀代码的自然表示。树概念编码含义从根开始。Root 解码开始 左边缘 $\rightarrow 0$ 和右边缘 $\rightarrow 1$ 。左边缘 Bit 0 右边缘 Bit 1 每个符号都存储在叶子中。叶完整符号 码字是根到叶的路径。叶子深度 代码长度 如果一个符号被放置在内部节点, 那么它的代码将是它下面的子树中所有代码的前缀。前缀代码树属性 在有效的前缀代码树中, 符号仅出现在叶子处。

优化目标: 每个字母的平均位数 (Optimization Objective: Average Bits per Letter)

代码 c 的每个字母平均位数定义为

$$\text{ABL}(c) = \sum_{x \in S} f_x |c(x)|.$$

在前缀码树 T 中, 代码长度等于叶子深度, 因此

$$\text{ABL}(T) = \sum_{x \in S} f_x \text{depth}_T(x).$$

其中 S 是符号集, f_x 是符号 x 的频率, $|c(x)|$ 是码字长度。

主要优化问题: 构建一个二叉前缀码 (Prefix Code) 树, 最小化加权外部路径长度 $\text{ABL}(T)$ 。

最优前缀代码树的结构 (Structure of an Optimal Prefix-Code Tree)

如果每个内部节点恰好有两个子节点, 则二叉树是满的。**命题** 最优前缀码对应的二叉树是满的。

证明想法。假设一个内部节点只有一个子节点。删除该冗余节点并将其子节点直接连接到其父节点。后果: 该子项下方的所有叶子都向根移动一层。它们的代码长度减少 1。其他代码长度没有增加。因此, 每个字母的平均位数会减少。这与最优性相矛盾。结论 最优前缀码树不能包含单子内部节点。

一个诱人的错误开始: Shannon-Fano (A Tempting False Start: Shannon-Fano)

Shannon-Fano 使用自上而下的策略:

- 按频率对符号进行排序。
- 将它们分成总频率几乎相等的两组。
- 分配前缀 0 和 1。
- 对两组进行递归。

为什么会失败 本地平衡不能保证最小全局加权路径长度。

经验教训 看似合理的贪婪规则仍然可能会错过最佳结果; 因此, 霍夫曼是自下而上构建的。

霍夫曼的贪婪选择 (Huffman's Greedy Choice)

主要观察结果是:

- 低频符号应该位于树的较深处。
- 满二叉树 (Full Binary Tree) 的最深层至少包含两个叶子。
- 在某个最优树中, 两个最不频繁的符号可以是兄弟。

贪心步骤: 选择两个最不频繁的符号 y 和 z , 并将它们合并为一个新的元符号 ω , 其中 $f_\omega = f_y + f_z$ 。

ω 表示两个子树为 y 和 z 的子树。

自下而上的贪心策略 重复合并两个最不频繁的节点。

霍夫曼算法 (Huffman Algorithm)

- 为每个符号创建一个叶节点。
 - 将所有节点插入到最小优先级队列中, 并以频率为关键字。
 - 重复提取频率最小的两个节点。
 - 创建一个新的内部节点, 其频率为两个提取频率之和。
 - 使提取的两个节点成为新节点的子节点。
 - 将新节点插回到优先级队列中。
 - 当只剩下一个节点时停止; 该节点是哈夫曼树的根。
- 复杂性: 没有优先级队列的情况下为 $O(n^2)$, 有最小堆的情况下为 $O(n \log n)$ 。

实现原理 使用最小优先级队列高效地重复查找两个最不频繁的节点。

示例：构建哈夫曼树 (Worked Example: Building a Huffman Tree)

符号频率为：

符号	a	e	k	l	u
频率	0.32	0.25	0.20	0.18	0.05

合并过程为：

$$\begin{aligned}0.05 + 0.18 &= 0.23, & 0.20 + 0.23 &= 0.43, \\0.25 + 0.32 &= 0.57, & 0.43 + 0.57 &= 1.\end{aligned}$$

经过这些合并后，频率为 1.00 的节点就是根。左/右选择会更改位串，但不会更改代码长度。一组有效的长度是 $|c(k)| = 2, |c(e)| = 2, |c(a)| = 2, |c(u)| = 3, |c(l)| = 3$ 。

重要事项 交换左子节点和右子节点会更改位模式，但不会更改平均代码长度。

为什么霍夫曼编码 (Huffman Coding) 是最优的？ (Why Is Huffman Coding Optimal?)

该证明结合了三个想法。

- 贪婪选择性质。存在最优前缀码树，其中两个最不频繁的符号是兄弟姐妹。
- 简化为更小的问题。将 y 和 z 合并为 ω 后，剩下的问题是另一个 Huffman 较小字母表上的实例。
- 平均长度关系。如果 T 是原始树， T' 是 y, z 合并到 ω 后的树，则

$$\text{ABL}(T) = \text{ABL}(T') + f_\omega, \quad \text{ABL}(T') = \text{ABL}(T) - f_\omega.$$

假设霍夫曼树 T 不是最优的，因此存在另一棵树 Z ，且 $\text{ABL}(Z) < \text{ABL}(T)$ 。选择这样一个 Z ，其中 y 和 z 是兄弟姐妹，并将它们合并到两棵树中以获得 Z' 和 T' 。那么 $\text{ABL}(Z') < \text{ABL}(T')$ ，与 T' 对于较小问题是最优的归纳假设相矛盾。结论哈夫曼编码构造了最优的前缀码。

总结 (Summary)

定长编码很简单，但忽略了符号频率。变长编码 (Variable-Length Encoding) 可以减少总位数。前缀码保证明确的解码。前缀码可以表示为二叉树。目标是 最小化 $\sum_{x \in S} f_x \text{depth}_T(x)$ 。

最优前缀码树已满。霍夫曼编码重复合并两个最不频繁的节点。使用优先级队列，霍夫曼编码的运行时间为 $O(n \log n)$ 。霍夫曼编码是一种具有形式最优性证明的贪心算法 (Greedy Algorithm)。

关键点霍夫曼编码是一个典型的例子，其中简单的贪婪规则导致全局最优解决方案。

课堂练习 (In-Class Exercise)

要求学生：符号频率 1 构建霍夫曼树。A 0.40 B 0.20 2 为所有符号分配二进制代码。C 0.15 3 计算每个字母的平均位数。D 0.15 E 0.10 4 将结果与定长编码进行比较。提示：首先合并两个最低频率，继续直到剩下一个节点，并记住左/右选择会更改代码，但不会更改代码长度。

讨论问题 当频率不均匀时，为什么霍夫曼编码优于固定长度编码？

第9周 分治法 (Divide and Conquer)

为什么要分治法？ (Why Divide and Conquer?)

许多算法问题都有简单的暴力解决方案。蛮力方法通常检查所有对或所有组合。这通常会导致二次时间： $O(n^2)$

对于大输入，二次时间很快就变得不切实际。分治法通常会将此类问题简化为： $O(n \log n)$

关键思想 不要直接解决一个大问题，而是将其分解为更小的子问题，递归地解决它们，并有效地组合它们的答案。

分治法模板 (The Divide-and-Conquer Template)

划分 将原始问题分解为更小的子问题。

征服 递归地解决每个子问题。

合并 将子问题的解合并为原问题的解。

对于许多平衡分治法算法，递推式为： $T(n) = 2T(n/2) + O(n)$

当问题分成大致相等的两半并且组合步骤花费线性时间时，总运行时间为： $O(n \log n)$

除法很容易；组合是关键 (Divide Is Easy; Combine Is the Key)

将输入分成两半通常很简单。递归地解决较小的问题是很自然的。主要挑战是组合步骤。

中心信息 分治法算法的质量取决于我们组合递归解决方案的效率。

示例：MergeSort：合并两个已排序的列表。计数反转：在计算交叉反转时进行合并。最接近的对：通过仅检查一小组候选点来组合。

排序作为起点 (Sorting as a Starting Point)

定义 给定 n 个元素，按升序重新排列它们。

为什么排序很重要 启用二分搜索。使重复检测更容易。帮助查找中位数和顺序统计数据。用作许多几何和数据分析问题的预处理。

排序不仅仅是一个独立的问题。一旦数据被排序，许多其他问题就会变得更容易。

归并排序 (MergeSort)：高级思想 (MergeSort: High-Level Idea)

合并排序遵循分治法的模板：

- 分割：将数组分割成两半。
- 征服：递归地对每一半进行排序。
- 合并：将已排序的两半合并为一个已排序的数组。 $T(n) = 2T(n/2) + O(n)$

示例：[8, 3, 5, 1, 9, 2] → [8, 3, 5] [1, 9, 2]

合并两个排序列表 (Merging Two Sorted Lists)

假设我们有两个排序列表： $A = [1, 4, 8, 10]$ $B = [2, 3, 7, 12]$

合并程序：在每个列表的前面保留一个指针。比较两个当前元素。将较小的元素移动到输出列表中。前进相应的指针。继续，直到复制所有元素。每个元素都被处理一次，因此合并需要： $O(n)$

归并排序运行时间 (MergeSort Running Time)

使用递归树视图：顶层： $O(n)$ 。下一层： $2 \cdot O(n/2) = O(n)$ 。再下一层： $4 \cdot O(n/4) = O(n)$ 。

每个级别的成本为 $O(n)$ 。级别数为： $\log_2 n$ 。

因此： $T(n) = O(n \log n)$

每层的关键计算成本乘以层数，即 $n \log n$ 。

归并排序总结 (MergeSort Summary)

MergeSort 说明了标准的分治法模型：两个大小为 $n/2$ 的子问题。两半的递归排序。线性时间合并。最终递推关系： $T(n) = 2T(n/2) + O(n)$

最终复杂度： $O(n \log n)$

接下来，我们使用相同的结构来解决一个显然不是排序问题的问题：计数反转。

计算反转 (Counting Inversions)

给定一个数组：[1,3,4,2,5]

反转是一对 (i, j) ，使得：

$i < j$ 但 $a_i > a_j$

在示例中，倒索引对是 $(2, 4)$ 和 $(3, 4)$ ，因为： $a_2 = 3 > a_4 = 2$ 。 $a_3 = 4 > a_4 = 2$ 。

翻转次数：2

反转衡量数组距排序的距离。

为什么要计算反转？ (Why Count Inversions?)

计算反转可以衡量两个排名之间的差异。应用：推荐系统。排名相似度。搜索结果比较。统计中的 Kendall Tau 距离。测量数组的“未排序”程度。示例：如果两个用户对同一首歌曲的排名相似，那么他们的排名之间的倒置次数就很小。如果反转的次数很多，他们的偏好就会有很大不同。

暴力计数 (Brute-Force Counting)

直接法：检查每对 (i, j) ，其中 $i < j$ 。检查是否 $a_i > a_j$ 。计算这样的对的数量。需要检查的数对数量为： $\binom{n}{2} = \frac{n(n-1)}{2}$ 。

运行时间： $O(n^2)$

太慢 对于大输入来说这太慢了。

我们可以在不单独检查每一对的情况下计算反转吗？

反演的分治法 (Divide-and-Conquer for Inversions)

将数组分成两半：左半部分和右半部分。每个反转都属于以下三个类别之一：1.左左反转：两个元素都在左半部分。2.右右反转：两个元素都在右半部分。3.分割反转：第一个元素在左半部分，第二个元素在右半部分。递归策略：递归地计算左-左反转。递归地计算右-右反转。计算组合步骤期间的分割反转。主要挑战 主要挑战是有效地计算分割反转。

计算合并期间的分割反转 (Counting Split Inversions During Merge)

假设两半都已排序。左：[1,5,8,10]

右：[2,6,9,12]

当当前右元素小于当前左元素时，它与左半部分的所有剩余元素形成反转。**示例** 当前左元素：5 当前右元素：2 由于左半部分已经排序，剩下的左元素5、8、10都大于2。因此，我们一次计算三个拆分反转：当前右侧元素与剩余左侧元素 5、8、10 中的每一个形成反转。

排序和计数 (Sort-and-Count)

递归函数返回两件事：1、翻转次数。

2. 输入数组的排序版本。

算法：基本情况：一个元素给出 0 次反转并且已经排序。拆分数组，然后递归排序并计算两半数组。合并已排序的两半并计算分割反转。返回总和：左左 + 右右 + 右分割

运行时间： $O(n \log n)$

一个有用的设计模式 (A Useful Design Pattern)

在计算反转中，递归不仅仅返回最终答案。它返回：（反转次数，排序数组）

为什么？因为父递归调用需要排序的数组来有效地计算拆分反转。设计课程 在分治算法中，递归调用通常会返回辅助信息，从而使组合步骤更快。

示例：

MergeSort 返回已排序的子数组。计数反转返回计数和排序子数组。最近的对将返回排序的点列表。

计数反转总结 (Counting Inversions Summary)

暴力算法检查所有对： $O(n^2)$

分治算法：将数组分成两半。递归地计算每一半中的反转。计算合并期间的拆分反转。组合步骤是线性的： $O(n)$

因此： $T(n) = 2T(n/2) + O(n) = O(n \log n)$

主要思想：使用排序结构一次计算多个反转。

最近的一对点 (Closest Pair of Points)

问题 给定平面上的 n 个点，找到具有最小欧氏距离的对。

两点之间的距离：

$$d(p, q) = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}.$$

暴力法：计算每对点之间的距离。返回最小距离。运行时间： $O(n^2)$

目标：设计一个 $O(n \log n)$ 分治算法。

为什么不分为四个象限？ (Why Not Divide into Four Quadrants?)

一个自然的想法：将平面分为四个区域。递归求解每个区域。问题：每个区域的点数可能高度不平衡。一个象限可能包含几乎所有的点，而其他象限则包含很少的点。重要原则 分治法需要平衡子问题。

结论：我们应该除以点数，而不是几何面积。

更好的除法步骤 (A Better Divide Step)

更好的策略：按 x 坐标对点进行排序。通过中值 x 坐标绘制一条垂直线 L 。将大约一半的点放在左边。将大约一半的点放在右侧。然后递归计算： δ_1 为左侧最近距离， δ_2 为右侧最近距离。

让： $\delta = \min(\delta_1, \delta_2)$ 。

此时，任何一半内都没有比 δ 更接近的对。

联合问题 (The Combine Problem)

递归后，最接近的对可能是：

1. 完全在左半部分。

2. 完全在右半边。3. 沿分割线 L 分割。前两种情况是递归解决的。剩下的问题是：我们如何有效地找到最接近的分割对？简单的方法：比较线上的所有对。这可能需要： $O(n^2)$

我们需要更快的合并步骤。

只有分界线附近的点才重要 (Only Points Near the Dividing Line Matter)

假设分裂对的距离小于 δ 。那么两个点必须位于分界线 L 的距离 δ 内。因此，我们只需要考虑垂直宽度条内的点： $|x_i - x_{\text{mid}}| < \delta$ 。

该区域称为 δ 带。带外的点不能形成比 δ 更近的分裂对。过滤想法 该条删除了许多不可能的候选者。

按 y 坐标对条带进行排序 (Sorting the Strip by y -Coordinate)

即使在地带内，也可能有很多点。下一步：按 y 坐标对条带点进行排序。观察：如果两个点的 y 坐标相差至少 δ ，则它们的欧几里德距离至少为 δ 。因此，对于每个点，我们只需要将其与后面 y 坐标接近的点进行比较。这为线性时间扫描准备了组合步骤。

打包论证 (The Packing Argument)

关键事实：按 y 坐标对条带进行排序后，每个点只需与下一个常数个点进行比较。不同的证明给出不同的常数（例如，常见的表示中为 7），但重要的事实是该数字是与 n 无关的固定常数。为什么？左侧已经没有比 δ 更近的对了。右侧已经没有比 δ 更近的对了。因此，同一侧的点不能堆积得太密集。在条带的一个小局部区域中，只能存在恒定数量的候选点。结论：每个点仅执行 $O(1)$ 比较，因此条带扫描需要 $O(n)$ 。

最近配对算法 (Closest Pair Algorithm)

算法概要：

1. 按 x 坐标对点进行排序。
2. 按中值 x 坐标分割点集。
3. 递归地在左半部分找到最接近的对。
4. 递归地在右半部分找到最接近的对。
5. 令 $\delta = \min(\delta_1, \delta_2)$ 。
6. 构建 δ 条带。
7. 按 y 排序顺序使用条带点。
8. 对于每个条带点，将其与下一个常量点进行比较。
9. 返回找到的最小距离。

最近对的运行时间 (Running Time of Closest Pair)

如果我们在每个递归级别从头开始对条带进行排序，则递归为： $T(n) = 2T(n/2) + O(n \log n)$

这给出： $O(n \log^2 n)$. 要达到 $O(n \log n)$ ：

维护按 x 坐标排序的点。维护按 y 坐标排序的点。在线性时间内合并这些排序列表，如合并排序。

然后： $T(n) = 2T(n/2) + O(n)$

因此： $T(n) = O(n \log n)$.

三个示例，一种模式 (Three Examples, One Pattern)

问题 除法 组合 运行时间 合并排序 将数组分成两半 合并两个排序列表 $O(n \log n)$ 计数反转 将数组分成两半 合并并计数拆分反转 $O(n \log n)$ 最近的对 按中值 x 坐标拆分 检查 δ 条带中的候选者 $O(n \log n)$

常见的结构是： $T(n) = 2T(n/2) + O(n)$

设计问题 (Design Questions)

设计分治法算法时，问：1. 我们应该如何划分问题？子问题是否平衡？它们的大小接近 $n/2$ 吗？

2. 每次递归调用应该返回什么？只有最终答案吗？或者额外的辅助信息？
3. 如何高效组合？合并步骤可以在 $O(n)$ 内完成吗？排序或几何结构有帮助吗？

设计原理 组合步骤通常决定算法的成功。

最后要点 (Final Takeaways)

主要信息 分治法是强大的，因为它将大问题转化为较小的结构化问题。

重要教训：

平衡除法使递归保持浅层。递归调用可能需要返回辅助信息。高效的组合步骤通常依赖于排序的结构或几何形状。

如果 $T(n) = 2T(n/2) + O(n)$ ，则 $T(n) = O(n \log n)$ 。

最后的陈述 除法很容易。高效联合收割机才是真正的关键。

第10周 快速乘法、卷积与快速傅里叶变换 (Integer Multiplication, Karatsuba, Convolution, and FFT)

如何更快地做乘法 (How to Multiply Faster)

1. 整数乘法能比小学的方法更快吗？
2. 为什么多项式乘法本质上是卷积 (Convolution) ？
3. FFT 如何将多项式乘法从 $O(n^2)$ 减少到 $O(n \log n)$ ？

改变表示方式，简化计算。

两个递归技巧Karatsuba：减少递归乘法。FFT：更改表示方式，使乘法是逐点的。

热身：复数乘法 (A Warm-up: Complex Multiplication)

$(a + bi)(c + di) = (ac - bd) + (ad + bc)i$ 直接方法需要计算 ac, bd, ad, bc ，共四次乘法。高斯技巧令

$$x = ac - bd, \quad y = (a + b)(c + d) - ac - bd = ad + bc,$$

于是 $(a + bi)(c + di) = x + yi$ 。

只需要计算 ac, bd 和 $(a + b)(c + d)$ 三个乘积。

4 次乘法 $\rightarrow$ 3 次乘法

分治法的复杂性 (Divide-and-Conquer Complexity)

$$T(n) = qT(n/2) + O(n) \implies T(n) = O(n^{\log_2 q}).$$

其中， q 是递归子问题的数量，每个子问题的规模为 $n/2$ ，拆分与合并成本为 $O(n)$ 。 $q = 4 \Rightarrow T(n) = O(n^2)$ 。

$$q = 3 \Rightarrow T(n) = O(n^{\log_2 3}) \approx O(n^{1.585}).$$

整数加法和乘法 (Integer Addition and Multiplication)

给定两个 n 位整数 a 和 b ：

整数加法 整数乘法

$a+b$ $a \times b$ 处理位和进位：小学方法： $\Theta(n)\Theta(n^2)$

小学乘法是最佳选择吗？

分治乘法 (Divide-and-Conquer Multiplication)

假设 n 是偶数；否则用前导零填充。将两个 n 位整数分成高半部分和低半部分：

$$a = 2^{n/2}a_1 + a_0, \quad b = 2^{n/2}b_1 + b_0.$$

$$ab = (2^{n/2}a_1 + a_0)(2^{n/2}b_1 + b_0).$$

$$ab = 2^n a_1 b_1 + 2^{n/2}(a_1 b_0 + a_0 b_1) + a_0 b_0.$$

这需要四次半长乘法：

$$a_1 b_1, a_1 b_0, a_0 b_1, a_0 b_0. \quad T(n) = 4T(n/2) + O(n)$$

为什么幼稚的分治法仍然是二次方的 (Why Naive Divide-and-Conquer Is Still Quadratic)

开始于： $T(n) = 4T(n/2) + O(n)$

级别 联合收割机总工作量 $O(n^1)$ $4 \cdot \frac{n}{2} = 2n$ $16 \cdot \frac{n}{4} = 4n$ $4^k \cdot \frac{n}{2^k} = 2^k n$

底层占主导地位，因此： $T(n) = \Theta(n^2)$

Karatsuba的主要观察 (Karatsuba's Key Observation)

中间项是： $a_1 b_0 + a_0 b_1$ 直接计算需要两次乘法。Karatsuba计算： $(a_1 + a_0)(b_1 + b_0)$ 展开为 $a_1 b_1 + a_1 b_0 + a_0 b_1 + a_0 b_0$ 。

因此：

$$a_1 b_0 + a_0 b_1 = (a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0.$$

Karatsuba算法 (Karatsuba Algorithm)

定义：

$$p_1 = a_1 b_1, \quad p_2 = a_0 b_0, \quad p_3 = (a_1 + a_0)(b_1 + b_0).$$

然后： $a_1 b_0 + a_0 b_1 = p_3 - p_1 - p_2$ 。

最终结果： $ab = 2^n p_1 + 2^{n/2}(p_3 - p_1 - p_2) + p_2$ 。

1. 将 a 、 b 分成高低两半。

2. 递归计算 p_1, p_2, p_3 。

3. 使用移位、加法和减法进行组合。

Karatsuba的运行时间 (Running Time of Karatsuba)

Karatsuba 使用三个半长递归乘法： $T(n) = 3T(n/2) + O(n)$

使用分治法的递归： $T(n) = O(n^{\log_2 3})$ 。

由于 $\log_2 3 \approx 1.585$ ，可得： $T(n) = O(n^{1.585}) < O(n^2)$

从整数乘法到多项式乘法 (From Integer Multiplication to Polynomial Multiplication)

现在考虑多项式乘法： $C(x) = A(x)B(x)$

如果 $A(x)$ 和 $B(x)$ 的次数均为 $n-1$ ，直接法需要 $O(n^2)$ 时间，目标是将其改进为 $O(n \log n)$ 。

基于直接FFT

工具：FFT，快速傅里叶变换 (Fourier Transform)

傅里叶变换：基本直觉 (Fourier Transform: Basic Intuition)

傅立叶分析表明，复杂信号可以表示为简单正弦曲线的和。 $y(t) = \sum_k c_k \sin(kt)$ 。

使用欧拉恒等式： $e^{ix} = \cos x + i \sin x$ 。

正弦和余弦可以使用复指数表示。

信号可以在另一个域中表示。

时域（Time Domain）与频域（Time Domain vs Frequency Domain）

可以在两个域中查看信号。

时域 频域（Frequency Domain） $y(t)$ 描述存在哪些频率。

描述信号如何随时间变化。 $y(t) = \sin(2\pi \cdot 697t) + \sin(2\pi \cdot 1209t)$ 。

2.2

时域：看起来很复杂的波。频域：697 Hz 和 1209 Hz 处的两个峰值。

卷积和过滤（Convolution and Filtering）

在数字滤波中，输入 $x[n]$ 通过滤波器 $h[n]$ ，产生输出 $y[n]$ 。 $y[n] = \sum_k x[k]h[n-k]$ 。

在频域： $Y(\omega) = X(\omega)H(\omega)$ 。

一个域内的卷积 $\Updownarrow$

变换域中的逐点乘法

DFT 和 FFT（DFT and FFT）

工具 时域 频域 含义 傅立叶级数 连续周期 离散周期信号 CTFT 连续 连续变换 DTFT 离散 连续离散序列 DFT 离散离散有限计算 FFT 与 DFT 相同 与 DFT 快速 DFT 算法

FFT 并不是一种不同的变换。FFT 是计算 DFT 的快速算法。

多项式：系数表示（Polynomial: Coefficient Representation）

多项式可以用其系数表示（Coefficient Representation）： $A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$ 。

添加很简单：计算 $A(x) + B(x)$ 需要 $O(n)$ 时间。乘积 $C(x) = A(x)B(x)$ 的系数为： $c_i = \sum_j a_j b_{i-j}$ 。

这就是卷积。直接计算需要： $O(n^2)$

多项式：点值表示（Polynomial: Point-Value Representation）

次数最多为 $n-1$ 的多项式由任意 n 个不同点处的值唯一确定。

$(x_0, A(x_0)), (x_1, A(x_1)), \dots, (x_{n-1}, A(x_{n-1}))$ 。

如果 $C(x) = A(x)B(x)$ ，则 $C(x_i) = A(x_i)B(x_i)$ 。

因此，点值形式的乘法是逐点的： $O(n)$

度详细信息 如果 A 和 B 的度数为 $n-1$ ，则 C 的次数最多为 $2n-2$ ，因此至少需要 $2n-1$ 个点。

为什么要在表示形式之间进行转换？（Why Convert Between Representations?）

系数表示 点值表示（Point-Value Representation） 自然存储 加法快 加法快 乘法快： $O(n)$ 乘法慢： $O(n^2)$ 需要转换

系数 FFT 点值逐点相乘 乘积的点值 逆 FFT 乘积系数

暴力转换太慢（Brute-Force Conversion Is Too Slow）

从系数到点值： $A(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1}$ 。

$A(x_0), A(x_1), \dots, A(x_{n-1})$ 。

使用霍纳方法：每个点需要 $O(n)$ 时间。

n 点总计： $O(n^2)$

从点值回到系数可以被视为求解范德蒙德系统。通用线性系统方法需要 $O(n^3)$ 。

我们可以选择特殊的点来使转换更快吗？

奇偶分解（Even-Odd Decomposition）

将多项式分解为偶次幂和奇次幂： $A(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_7x^7$ 。 $A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + a_6x^3$ 。

$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + a_7x^3$ 。

然后： $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$ 。

为什么配对点有帮助（Why Paired Points Help）

由偶奇分解得： $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$ 。 $A(-x) = A_{\text{even}}(x^2) - xA_{\text{odd}}(x^2)$ 。

因此， $A(x)$ 和 $A(-x)$ 共享子问题 $A_{\text{even}}(x^2)$ 与 $A_{\text{odd}}(x^2)$ 的计算。

$A(1) = A_{\text{even}}(1) + A_{\text{odd}}(1)$ ， $A(i) = A_{\text{even}}(-1) + iA_{\text{odd}}(-1)$ ， $A(-1) = A_{\text{even}}(1) - A_{\text{odd}}(1)$ ， $A(-i) = A_{\text{even}}(-1) - iA_{\text{odd}}(-1)$ 。

Unity 和 DFT 的根源 (Roots of Unity and DFT)

定义: 如果 $x^n = 1$,

那么 x 是单位的 n 次方根。让: $\omega = e^{2\pi i/n}$.

那么所有的 n 次单位根 (Roots of Unity) 是: $1, \omega, \omega^2, \dots, \omega^{n-1}$.

使用正指数约定, 系数向量 $a_0, \dots, a_{n-1}$ 的 DFT 为: $y_k = A(\omega^k)$.

DFT: 系数 $\rightarrow$ 点值

FFT 递归 (FFT Recursion)

目标: $A(\omega^0), A(\omega^1), \dots, A(\omega^{n-1})$.

用途: $A(x) = A_{\text{even}}(x^2) + xA_{\text{odd}}(x^2)$.

令 $\nu = \omega^2$, 则 ν 是一个 $n/2$ 次单位根。递归计算:

$$E_k = A_{\text{even}}(\nu^k), \quad D_k = A_{\text{odd}}(\nu^k).$$

对于 $0 \leq k < n/2$, 合并为: $A(\omega^k) = E_k + \omega^k D_k, A(\omega^{k+n/2}) = E_k - \omega^k D_k$.

FFT算法 (FFT Algorithm)

FFT (a, n) : 若 $n = 1$, 则返回 a ; 否则计算 $E = \text{FFT}(a_0, a_2, \dots, a_{n-2})$ 和 $D = \text{FFT}(a_1, a_3, \dots, a_{n-1})$. 对 $k = 0, \dots, n/2 - 1$, 令 $w = \exp(2\pi i k/n)$, 并计算 $y_k = E_k + w D_k, y_{k+n/2} = E_k - w D_k$, 最后返回 y .

E 和 D: 偶数和奇数次多项式的 DFT。组合步骤是蝶形操作。

FFT 的运行时间 (Running Time of FFT)

其递归式为: $T(n) = 2T(n/2) + O(n)$

共有 $\log_2 n$ 层递归。在每个级别, 总组合成本为: $O(n)$

因此: $T(n) = O(n \log n)$

假设 n 是 2 的幂。如果不是, 则用零填充到 2 的下一个幂。

位反转 (Bit-Reversal) 和迭代 FFT (Bit-Reversal and Iterative FFT)

递归 FFT 自然会按位反转顺序访问系数。当 $n = 8$ 时, 原始顺序为:

0、1、2、3、4、5、6、7

递归顺序变为: 0, 4, 2, 6, 1, 5, 3, 7

二进制形式: 000、100、010、110、001、101、011、111

实现思路:

- 通过位反转对输入重新排序。
- 执行迭代蝶形合并。

逆 FFT (Inverse FFT)

FFT 将系数映射到点值: $a \rightarrow y$

对于多项式乘法, 我们还需要恢复系数:

$y \rightarrow a$

这就是逆FFT。关键事实 逆 FFT 与 FFT 几乎相同:

- 将单位根替换为 $\omega^{-1} = e^{-2\pi i/n}$ 。
- 将所有输出除以 n 。

$$\text{IFFT}(y) = \frac{1}{n} \text{FFT}_{\omega^{-1}}(y).$$

FFT 多项式乘法 (Polynomial Multiplication with FFT)

目标: $C(x) = A(x)B(x)$, 其中 A 和 B 的度数最多为 $n-1$ 。

- 用零填充。由于 $\deg C \leq 2n-2$, 至少需要 $2n-1$ 个点。通常垫到 取满足 $m \geq 2n-1$ 的二次幂。
- 计算 FFT。

$$A \mapsto A(\omega^0), \dots, A(\omega^{m-1}), \quad B \mapsto B(\omega^0), \dots, B(\omega^{m-1}).$$

3.逐点相乘。 $C(\omega^k) = A(\omega^k)B(\omega^k)$.

- 使用逆FFT。

$$C(\omega^k) \mapsto c_0, c_1, \dots, c_{2n-2}. \quad O(m \log m) + O(m) + O(m \log m) = O(m \log m) = O(n \log n)$$

总结 (Summary)

1. 小学整数乘法: $O(n^2)$ 。
2. Karatsuba乘法: $T(n) = 3T(n/2) + O(n) = O(n^{1.585})$
3. 系数形式的多项式乘法就是卷积: $c_i = \sum_j a_j b_{i-j}$. 直接计算需要 $O(n^2)$ 时间。
 4. 在点值形式中, 多项式乘法是逐点的: $O(n)$ 。
 5. FFT 在系数和点值形式之间进行转换, 时间复杂度为 $O(n \log n)$ 。
 6. 因此多项式乘法可以在 $O(n \log n)$ 内完成。

正确的表示使计算更加容易。

第11-12周 动态规划 (Dynamic Programming)

主要思想 (Main idea)

动态规划 (Dynamic Programming) 是一种通过求解并复用相互重叠的子问题来解决问题的算法设计技术, 常用于优化、计数和判定问题。

它将一个大问题分解为重叠的子问题。它存储子问题的解决方案并重用它们。它从最佳解决方案到较小问题构建较大问题的最佳解决方案。

章节实例加权区间调度 (Weighted Interval Scheduling); 分段最小二乘 (Segmented Least Squares) 法; 背包问题 (Knapsack Problem); RNA二级结构 (RNA Secondary Structure); 序列比对 (Sequence Alignment); 线性空间对齐。

三种算法范式 (Three Algorithmic Paradigms)

范式 主要思想 子问题关系 示例 贪婪 做出局部最优选择 不解决所有子问题 区间调度 (Interval Scheduling) 分治法 分割、独立求解、独立子问题 归并排序 组合 动态规划-求解和重用子问题 重叠子问题 (Overlapping Subproblems) 背包、对齐求解

关键视图 动态编程可以看作是递归加内存, 或者按照依赖顺序填充表。

何时使用动态规划 (When to Use Dynamic Programming)

两个基本性质 1 最优子结构 (Optimal Substructure): 最优解包含子问题的最优解。

2. 重叠子问题: 相同的子问题在朴素递归解决方案中重复出现。

基本 DP 配方 1 定义状态。

2. 推导递推式。
3. 确定边界条件。
4. 选择计算顺序。
5. 如果需要, 恢复最优解。

加权区间调度 (Weighted Interval Scheduling)

给定 n 个工作。每个工作 j 有: 开始时间 s_j 、结束时间 f_j 、值或重量 v_j 。如果两个作业的时间间隔不重叠, 则它们是兼容的。目标选择相互兼容的工作的最大值子集。

未加权版本最大化了作业数量。加权版本使总价值最大化。

为什么贪婪会失败 (Why Greedy Fails)

对于非加权间隔调度: 按最早完成时间排序给出了最佳的贪心算法 (Greedy Algorithm)。对于加权间隔调度: 最早完成时间可以选择低价值的工作。较晚或较长的工作可能具有更大的价值。

想法示例 工作 A: 时间短, 提前完成, 价值 1。作业 B: 长, 重叠 A, 值 999。

结论 当工作具有任意值时, 本地决策并不可靠。

按完成时间排序并定义 $p(j)$ (Sort by Finish Time and Define $p(j)$)

按非递减完成时间对作业进行排序: $f_1 \leq f_2 \leq \dots \leq f_n$ 。

定义: $p(j)$ 是满足 $i < j$ 且作业 i 与作业 j 兼容的最大索引 i ; 若不存在, 则令 $p(j) = 0$ 。

如果选择了作业 j , 则此前选择的作业只能来自 $1, \dots, p(j)$ 。作业 $p(j) + 1, \dots, j - 1$ 与作业 j 冲突。

状态定义 (State Definition)

定义: $\text{OPT}(j)$ 表示仅使用作业 $1, 2, \dots, j$ 时可获得的最大总价值。

目标: $\text{OPT}(n)$ 。

为什么此状态有效 作业按完成时间排序。问题可以从前 j 个作业开始逐步构建。最后一个作业 j 创建了一个干净的包含或排除决策。

二元选择递归 (Binary Choice Recurrence)

对于作业 j , 最优解有两种可能。

包括工作 j 排除工作 j

$$\text{包含 } j: v_j + \text{OPT}(p(j)), \quad \text{排除 } j: \text{OPT}(j - 1).$$

因此:

$$\text{OPT}(j) = \max \{v_j + \text{OPT}(p(j)), \text{OPT}(j-1)\}.$$

边界条件: $\text{OPT}(0) = 0$.

朴素递归 (Naive Recursion)

直接递归实现遵循递归:

```
Compute-Opt(j):
    if j = 0:
        return 0
    return max(v_j + Compute-Opt(p(j)),
              Compute-Opt(j - 1))
```

这在数学上是正确的, 但计算效率低下。问题 相同的子问题被重新计算多次。

为什么朴素递归可以是指数级的 (Why Naive Recursion Can Be Exponential)

在某些情况下, 我们可能有: $p(j) = j - 2$.

那么递归调用结构类似于: $T(j) = T(j-1) + T(j-2) + O(1)$.

这就像斐波那契数列一样增长。OPT(k)通过不同的递归路径重复计算。

解决方案对每个子问题计算一次并存储其结果。

记忆化 (Memoization)

使用数组 M 来存储计算值。初始化 $M[0] = 0$ 并将所有其他条目初始化为空。

```
M-Compute-Opt(j):
    if j = 0:
        return 0
    if M[j] 为空:
        M[j] = max(v_j + M-Compute-Opt(p(j)),
                  M-Compute-Opt(j - 1))
    return M[j]
```

相同的递归逻辑。每个状态 j 仅计算一次。稍后的调用返回缓存的值。

自下而上计算 (Bottom-Up Computation)

按 j 的递增顺序填充表, 而不是递归: $M[0] = 0$

```
for j = 1, ..., n:
    M[j] = max(v_j + M[p(j)], M[j - 1])
```

答: $M[n]$ 。

为什么这个顺序有效: $p(j) < j$ 且 $j-1 < j$, 因此所有依赖关系都已计算。

恢复解决方案 (Recovering the Solution)

DP 表给出了最优值。若 $v_j + M[p(j)] > M[j-1]$, 则选择作业 j, 并从 $p(j)$ 继续; 否则从 $j-1$ 继续。

```
Find-Solution(j):
    if j = 0:
        return
    if v_j + M[p(j)] > M[j - 1]:
        输出 j
        Find-Solution(p(j))
    else:
        Find-Solution(j - 1)
```

总结: 加权区间调度 (Summary: Weighted Interval Scheduling)

状态 $\text{OPT}(j)$ 表示作业 $1, \dots, j$ 的最大总价值。

递归和边界

$$\text{OPT}(j) = \max \{v_j + \text{OPT}(p(j)), \text{OPT}(j-1)\}, \quad \text{OPT}(0) = 0.$$

作业排序: $O(n \log n)$. 计算所有 $p(j)$: $O(n \log n)$. 动态规划计算: $O(n)$.

建模模式 二元选择: 包含或排除最后一项。

分段最小二乘法 (Segmented Least Squares)

输入: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, 其中 $x_1 < x_2 < \dots < x_n$ 。

普通最小二乘法用一条线拟合所有点: $y = ax + b$ 。

但许多数据集最好由几条线段表示。问题 我们如何平衡准确性和简单性?

准确性与简单性 (Accuracy vs. Simplicity)

优化目标为 $E + cL$ 。

其中, E 是所有线段的总平方误差, L 是线段数, $c > 0$ 是每增加一段的固定惩罚。

解释 较小的E: 更适合。较小的L: 更简单的模型。较大的c: 有利于较少的段。较小的c: 允许更多段。

预计算段错误 (Precompute Segment Errors)

定义:

$e(i, j)$ 表示用一条直线拟合点 $p_i, \dots, p_j$ 时的最小平方误差。

对于任意区间 $i, \dots, j$, $e(i, j)$ 表示: 含义 如果这些点形成一条线段, 那么最佳的拟合误差是多少?

然后DP只需要决定片段的开始和结束位置。

状态定义 (State Definition)

定义: $\text{OPT}(j)$ 表示拟合点 $p_1, \dots, p_j$ 的最小成本。

目标: $\text{OPT}(n)$ 。

要计算 $\text{OPT}(j)$, 请问: 最后的决定 最后一段从哪里开始?

假设最后一段是 $p_i, p_{i+1}, \dots, p_j$, 则点 $p_1, \dots, p_{i-1}$ 必须已经被最优拟合。

多路选择递归 (Multiway Choice Recurrence)

如果最后一段是 $i, \dots, j$, 则总成本为 $\text{OPT}(i-1) + e(i, j) + c$ 。

组件: $\text{OPT}(i-1)$: 最后一段之前的最优成本。e(i, j): 最后一段的拟合误差。c: 添加此段的惩罚。因此:

$$\text{OPT}(j) = \min_{1 \leq i \leq j} \{ \text{OPT}(i-1) + e(i, j) + c \}.$$

边界: $\text{OPT}(0) = 0$ 。

为了解决方案恢复, 存储每个j的最小化起始索引i。

算法和复杂性 (Algorithm and Complexity)

算法:

```
M[0] = 0
for j = 1, ..., n:
    M[j] = infinity
    for i = 1, ..., j:
        candidate = M[i-1] + e(i, j) + c
        if candidate < M[j]:
            M[j] = candidate
            first[j] = i
```

两个阶段:

- 计算所有段误差e(i, j)。
- 填充DP数组M[j]。

$O(n^2)$ 间隔; 朴素预处理 $O(n^3)$ 。预处理后的动态规划: $O(n^2)$ 。

通过累积统计, 每个 e(i, j) 的复杂度为 $O(1)$, 总体上为 $O(n^2)$ 。

总结: 分段最小二乘法 (Summary: Segmented Least Squares)

状态 $\text{OPT}(j)$ 表示拟合 $p_1, \dots, p_j$ 的最小成本。

递归和边界

$$\text{OPT}(j) = \min_{1 \leq i \leq j} \{ \text{OPT}(i-1) + e(i, j) + c \}, \quad \text{OPT}(0) = 0.$$

建模模式多路选择: 选择最后一段的开始。

加权间隔调度有二元最终选择; 分段最小二乘法有许多可能的最后一段开始。

背包问题 (Knapsack Problem)

输入：n 项。项目 i 的重量 $w_i > 0$ 且值 $v_i > 0$ 。背包容量（Capacity）为 W。

目标选择总重量最多为 W 且总价值最大的物品。

限制：每个项目最多只能选择一次。这就是 0/1 背包问题。

贪婪并不是最优的 (Greedy Is Not Optimal)

可能的贪心规则：选择价值最大的项目。选择价值与重量比最大的物品。首先选择最轻的物品。这些都不能保证是最佳的。原因 容量会在选择之间产生依赖性。局部有吸引力的项目可能会阻碍更好的项目组合。

结论 我们必须比较不同项目前缀和容量限制的最佳值。

错误的开始 (A False Start)

尝试定义： $\text{OPT}(i)$ 表示仅使用物品 $1, \dots, i$ 时的最大价值。

这还不够。如果未选择第 i 项，则使用 $\text{OPT}(i - 1)$ 。但如果选择了第 i 项，我们需要知道：已经使用了多少容量？项目 i 是否有足够的剩余容量？选择项目 i 后剩余容量是多少？

教训 状态必须包括容量。

添加容量变量 (Add a Capacity Variable)

定义： $\text{OPT}(i, w)$ 表示容量为 w 时，从物品 $1, \dots, i$ 中可获得的最大总价值。

目标： $\text{OPT}(n, W)$ 。

对于项目 i：不要选择项目 i $\text{OPT}(i - 1, w)$ 。

若 $w_i \leq w$ ，选择物品 i 的候选值为 $v_i + \text{OPT}(i - 1, w - w_i)$ 。

背包循环 (Knapsack Recurrence)

边界： $\text{OPT}(0, w) = 0$ 。

如果物品 i 太重：

$$w_i > w \Rightarrow \text{OPT}(i, w) = \text{OPT}(i - 1, w).$$

否则：

$$\text{OPT}(i, w) = \max\{\text{OPT}(i - 1, w), v_i + \text{OPT}(i - 1, w - w_i)\}.$$

排除项目 i。包含第 i 项并求解剩余容量子问题。

自下而上表 (Bottom-Up Table)

创建表 $M[i, w]$ ，其中：

$M[i, w]$ = 使用容量为 w 的前 i 个项目的最佳值。

初始化： $M[0, w] = 0, M[i, 0] = 0$ 。

```
for i = 1, ..., n:
    for w = 1, ..., W:
        if w_i > w:
            M[i, w] = M[i - 1, w]
        else:
            M[i, w] = max(M[i - 1, w],
                          v_i + M[i - 1, w - w_i])
```

答案： $M[n, W]$ 。

示例：一个表条目 (Example: One Table Entry)

项目：

项目 价值 重量 1 1 1 2 6 2 3 18 5 4 22 6 5 28 7 容量： $W = 11$ 。

示例：计算 $M[3, 7]$ 。不选择物品 3： $M[2, 7] = 7$ 。选择物品 3： $18 + M[2, 2] = 18 + 6 = 24$ 。

因此： $M[3, 7] = 24$ 。

恢复最佳物品 (Recovering the Optimal Items)

举个例子： $M[5, 11] = 40$ 。

最佳项目集： $\{3, 4\}$ 。

总重量： $5 + 6 = 11$ 。

总价值： $18 + 22 = 40$ 。

回溯规则 如果 $M[i, w] = M[i - 1, w]$ ，则项目 i 未被选择。否则，选择项目 i，并继续使用容量 $w - w_i$ 。

运行时间 (Running Time)

桌子尺寸: $(n+1)(W+1)$ 。

每个条目花费 $O(1)$ 时间。**复杂性** 时间 = $\Theta(nW)$, 空间 = $\Theta(nW)$ 。

重要提示 这是伪多项式时间。

原因: 运行时间取决于数值 W , 而不仅仅是用于编码 W 的位数。

RNA二级结构 (RNA Secondary Structure)

RNA 分子是一个字符串: $\{A, C, G, U\}$ 。

RNA 是单链的, 可以自行折叠形成碱基对。沃森-克里克互补对:

A-U、U-A、C-G、G-C。

算法目标 给定一个 RNA 序列, 找到一个有效的二级结构, 使碱基对的数量最大化。

有效性规则 (Validity Rules)

有效的 RNA 二级结构必须满足:

- Watson-Crick 对: 只有互补碱基才可以配对。
- 没有急转弯: 如果 (b_i, b_j) 是一对, 则 $i < j - 4$ 。
- 非交叉对: 我们不能有两个对 (b_i, b_j) 和 (b_k, b_l) 其中 $i < k < j < l$ 。

目标

最大化有效碱基对的数量。

为什么前缀 DP 不够 (Why Prefix DP Is Not Enough)

第一次尝试可能会定义:

$OPT(j)$ 表示前缀 $b_1, \dots, b_j$ 中最多可形成的碱基对数量。

问题: 如果 b_n 与某个 b_t 配对, 则序列将分为: $b_1, \dots, b_{t-1}$

和 $b_{t+1}, \dots, b_{n-1}$ 。

第二部分不是前缀。结论 我们需要跨区间的子问题, 而不仅仅是前缀。

间隔 DP 状态 (Interval DP State)

定义: $OPT(i, j)$ 表示子串 $b_i, \dots, b_j$ 中最多可形成的碱基对数量。

目标: $OPT(1, n)$ 。

边界条件: 如果间隔太短, $i \geq j - 4$,

那么 $OPT(i, j) = 0$ 。

原因: 无急转弯规则会阻止在如此短的间隔内进行有效配对。

RNA 递推关系 (RNA Recurrence)

考虑区间 $i, \dots, j$ 中的最后一个碱基 b_j 。情况 1: b_j 不配对, 候选值为 $OPT(i, j - 1)$ 。

情况 2: b_j 与某个 b_t 配对。要求 $i \leq t < j - 4$, 且 b_t 与 b_j 互补。候选值为

$1 + OPT(i, t - 1) + OPT(t + 1, j - 1)$ 。

因此:

$$OPT(i, j) = \max \left\{ OPT(i, j - 1), \max_{\substack{i \leq t < j - 4 \\ b_t \text{ 与 } b_j \text{ 互补}}} [1 + OPT(i, t - 1) + OPT(t + 1, j - 1)] \right\}.$$

自下而上的间隔 (Bottom-Up over Intervals)

先计算较短的间隔, 然后再计算较长的间隔。原因: $OPT(i, j)$ 取决于较小的间隔。这里的 span 是指 $j - i$ 。

```
for span = 5, ..., n - 1:
    for i = 1, ..., n - span:
        j = i + span
        计算 OPT(i, j)
```

计算 $OPT(i, j)$

复杂性 区间数量: $O(n^2)$ 。

配对位置 t 的选择: 每个间隔 $O(n)$ 。总时间: $O(n^3)$ 。

序列比对 (Sequence Alignment)

问题：

两个字符串或生物序列有多相似？

示例：

发生 发生

我们可以插入间隙以更好地对齐字符串。两种成本：不匹配：两个不同的字符对齐。间隙：字符与空白对齐。目标：找到最小总成本的对齐方式。

成本模型 (Cost Model)

让： δ ：间隙扣分。 α_{pq} ：将字符 p 与字符 q 对齐的成本。总对齐成本：

错配成本之和+差距惩罚之和。

应用：文件比较，例如Unix diff。语音识别。DNA、RNA 和蛋白质序列比对。

对齐的定义 (Definition of Alignment)

鉴于：

$$X = x_1x_2\cdots x_m, \quad Y = y_1y_2\cdots y_n.$$

对齐是一组有序对 $x_i - y_j$ ，满足：每个字符最多出现在一个有序对中，并且各对不交叉。非交叉条件：

如果 $i < i'$ ，则对应的配对索引必须满足 $j < j'$ 。

含义 对齐必须保留两个字符串的原始顺序。

状态定义 (State Definition)

定义：OPT(i, j) 表示对齐 $x_1, \dots, x_i$ 与 $y_1, \dots, y_j$ 的最小成本。

目标：OPT(m, n)。

边界条件：

$$\text{OPT}(i, 0) = i\delta, \quad \text{OPT}(0, j) = j\delta.$$

说明：与空前缀对齐的非空前缀必须仅使用间隙。

最后一步的三种可能性 (Three Possibilities for the Last Step)

要计算 OPT(i, j)，考虑最后的对齐决策：将 x_i 与 y_j 对齐，成本为 $\alpha_{x_iy_j} + \text{OPT}(i-1, j-1)$ ；将 x_i 与间隙对齐，成本为 $\delta + \text{OPT}(i-1, j)$ ；将 y_j 与间隙对齐，成本为 $\delta + \text{OPT}(i, j-1)$ 。因此：

$$\text{OPT}(i, j) = \min \{ \alpha_{x_iy_j} + \text{OPT}(i-1, j-1), \delta + \text{OPT}(i-1, j), \delta + \text{OPT}(i, j-1) \}.$$

算法和复杂性 (Algorithm and Complexity)

```
for i = 0, ..., m:
    M[i, 0] = i * delta
for j = 0, ..., n:
    M[0, j] = j * delta
for i = 1, ..., m:
    for j = 1, ..., n:
        M[i, j] = min(alpha[x_i, y_j] + M[i-1, j-1],
                      delta + M[i-1, j],
                      delta + M[i, j-1])
```

答案：M[m,n]。时间 = $\Theta(mn)$ ， 空间 = $\Theta(mn)$ 。

限制 对于很长的生物序列，二次空间表可能太大。

从二次空间到线性空间 (From Quadratic Space to Linear Space)

标准递推关系是：

$$M[i, j] = \min \{ \alpha_{x_iy_j} + M[i-1, j-1], \delta + M[i-1, j], \delta + M[i, j-1] \}.$$

从左到右计算第 i 行时，每个单元格只需要：上一行：M[i-1, j-1] 和 M[i-1, j]；紧邻左侧的当前行值：M[i, j-1]。因此，要计算最优值，我们只需要：

上一行+当前行。

值计算的空间： $O(n) \subseteq O(m+n)$ 。

滚动数组 (Rolling Arrays) 以获得最佳值 (Rolling Arrays for the Optimal Value)

使用两个长度为 $n + 1$ 的数组: $\text{prev}[j]$ 存储前一个 DP 行; $\text{curr}[j]$ 存储当前正在计算的行。 $\text{prev}[j] = j\delta, \quad j = 0, \dots, n.$

```
for i = 1, ..., m:
    curr[0] = i * delta
    for j = 1, ..., n:
        curr[j] = min(alpha[x_i, y_j] + prev[j - 1],
                      delta + prev[j],
                      delta + curr[j - 1])
    交换(prev, curr)
```

最终答案为 $\text{prev}[n]$ 。空间复杂度为 $O(n)$; 如果将较短序列作为列, 则为 $O(\min\{m, n\})$ 。

价值与一致性 (Value vs. Alignment)

滚动数组可以计算最优对齐成本, 但恢复完整对齐通常需要回溯 DP 表, 确定每个单元格选择的前驱: 对角线表示将 x_i 与 y_j 对齐; 向上表示将 x_i 与间隙对齐; 向左表示将 y_j 与间隙对齐。只保留两行会丢弃表的大部分内容, 因此问题是如何仅用线性空间恢复实际的最优对齐。

对齐作为最短路径 (Alignment as a Shortest Path)

将 DP 表表示为网格图。从 $(0, 0)$ 到 (m, n) 的路径对应一个对齐: 对角线移动表示将 x_i 与 y_j 对齐, 垂直移动表示将 x_i 与间隙对齐, 水平移动表示将 y_j 与间隙对齐。路径成本等于对齐成本。

最佳对齐 = 从 $(0, 0)$ 到 (m, n) 的最短路径。

找到最佳路径与中间相交的位置 (Find Where the Optimal Path Crosses the Middle)

选择中间列 $j^* = \lfloor \frac{n}{2} \rfloor$ 。

定义前向成本: $f(i, j^*)$ 是从 $(0, 0)$ 到 (i, j^*) 的最短路径成本。

定义向后成本: $g(i, j^*)$ 是从 (i, j^*) 到 (m, n) 的最短路径成本, 可通过在反向后缀上运行相同的 DP 计算。两个向量都可使用滚动数组计算。对每一行 i , 经过 (i, j^*) 的最短路径成本为 $f(i, j^*) + g(i, j^*)$ 。

选择分割点 (Choosing the Split Point)

查找:

$$q = \arg \min_{0 \leq i \leq m} \{f(i, j^*) + g(i, j^*)\}.$$

那么最优对齐路径经过 (q, j^*) 。

为什么? 从 $(0, 0)$ 到 (m, n) 的每条路径都必须穿过中间列。最优交叉点使到达该点的成本与从该点到终点的成本之和最小。路径经过 (q, j^*) 并不意味着 x_q 与 y_{j^*} 对齐, 只表示最优路径经过该网格点。

通过分治法恢复对齐 (Recover the Alignment by Divide and Conquer)

找到分割点 (q, j^*) 后, 递归求解两个子问题: $X[1 : q]$ 与 $Y[1 : j^*]$, 以及 $X[q + 1 : m]$ 与 $Y[j^* + 1 : n]$ 。这些索引范围描述了网格点 (q, j^*) 前后的两条子路径。在每次递归调用时:

- 1. 使用滚动数组计算前向成本;
- 2. 使用滚动数组计算向后成本;
- 3. 找到最佳的中间交叉点;
- 3. 递归解决两个较小的对齐问题。

复杂性 内存使用: $O(m + n)$ 。运行时间: $O(mn)$ 。

本章的动态规划模式 (Dynamic Programming Patterns in This Chapter)

问题 DP 建模模式 加权间隔调度 二元选择: 包括或排除最后一个作业 分段最小二乘法 多路选择: 最后一个分段从哪里开始? Knapsack 添加新变量: 容量 RNA 二级结构间隔 DP 序列比对 二维前缀 DP 线性空间比对 DP 加分治法

主要课程动态规划不是要记住公式。它是关于定义正确的子问题并从最终决策中推导出递归。

第13周 最短路径、Bellman-Ford与距离向量协议 (Shortest Paths, Bellman-Ford, and Distance Vector Protocols)

最短路径问题 (Shortest Path Problem)

给定一个有向图 (Directed Graph) $G = (V, E)$,

每条有向边 $(v, w) \in E$ 都有成本 c_{vw} 。目标是找到从源节点 s 到目标节点 t 的最小成本路径。对于路径 $P = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$,

它的成本是 $c(P) = \sum_{i=0}^{k-1} c_{v_i v_{i+1}}$ 。

重要区别与标准 Dijkstra 设置不同, 边缘成本可能为负。负成本可以模拟利润或收益, 例如有利的金融交易。

示例: 为什么负边沿很重要 (Example: Why Negative Edges Matter)

考虑带边的有向图

$s \rightarrow u \rightarrow v \rightarrow t, s \rightarrow t$ 。

令 $c_{su} = 2, c_{uv} = 3, c_{vt} = -6, c_{st} = 1$ 。

直接路径有成本 $c(s \rightarrow t) = 1$ 。

但更长的路是有代价的 $c(s \rightarrow u \rightarrow v \rightarrow t) = 2 + 3 - 6 = -1$ 。

结论 具有更多边的路径更便宜。负边打破了这样的直觉：路径越长，成本必然会越高。

为什么 Dijkstra 算法会失败 (Why Dijkstra’s Algorithm Can Fail)

Dijkstra算法选择暂定距离最小的未解决节点并将其永久修复。仅当所有边权重均为非负时，此永久标签步骤才是安全的。对于非负边，首先经过另一个未解决节点的路径随后不会变得更便宜。有了负边，现在看起来最优的节点以后可能会通过负边得到改进。

在前面的例子中，扫描完 s 中的边缘后，t 的暂定距离为 1，而更好的路径

$s \rightarrow u \rightarrow v \rightarrow t$

总成本为 $2 + 3 - 6 = -1$ 并且仅在到达 v 后才被发现。结论 Dijkstra 的永久标签步骤对于具有负边权重的图无效。

为什么简单的重新加权会失败 (Why Simple Reweighting Fails)

一个诱人的想法是为每个边权重添加一个常数 C，以便所有权重都变为非负值。考虑两条 s 到 t 路径：

路径 A： $2 + 2 = 4$ ， 路径 B： $3 + (-3) + 3 = 3$ 。

最初，路径 B 较短。如果我们在每条边上加 3，那么

路径 A： $5 + 5 = 10$ ， 路径 B： $6 + 0 + 6 = 12$ 。

现在路径 A 更短了。原因 如果路径有 k 条边，则将 C 添加到每条边会增加 Ck 的成本。具有不同边数的路径受到不同的惩罚。

负成本周期 (Negative-Cost Cycles)

定义 如果 $c(W) < 0$ ，则循环 W 是负成本循环。

例如，边成本为 7, -4, -6 的循环的总成本 $7 - 4 - 6 = -3$ 。

如果从 s 到 t 的路径可以到达并使用此循环，则可以重复绕过它：1次：-3，2次：-6，k次：-3k。

结论 如果负成本环位于从 s 到 t 的某个可达路径上，则最短路径不存在。

当最短路径确实存在时 (When Shortest Paths Do Exist)

观察 如果到 t 的路线上不存在可达到的负成本循环，则每个有限最短距离都可以通过简单路径 (Simple Path) 实现。

简单路径不重复任何顶点。如果路径重复一个顶点，则重复部分形成一个循环。如果该循环具有非负成本，则删除它不会增加路径成本。如果它的成本为负并且仍然可以到达 t，则距离没有明确定义。

在 $n = |V|$ 的图中顶点，每条简单路径至多有 $n - 1$ 条边。

结论 当最短路径被明确定义时，考虑最多具有 $n - 1$ 条边的路径就足够了。

动态规划 (Dynamic Programming) 公式 (Dynamic Programming Formulation)

将 $OPT(i, v)$ 定义为使用最多 i 个边从节点 v 到目标节点 t 的路径的最小成本。这是一个面向目标的定义：对于一个固定的目的地 t，我们计算从每个节点 v 到 t 的最佳距离。基本情况

$OPT(0, t) = 0, \quad OPT(0, v) = \infty \quad \text{对所有 } v \neq t.$

在零边的情况下，只有 t 可以到达自身。

参数 i 控制路径中允许的最大边数。

贝尔曼-福特递归 (The Bellman-Ford Recurrence)

对于 $i \geq 1$ ，考虑使用最多 i 个边的最短 v 到 t 路径。如果它最多使用 $i - 1$ 条边，则其成本为 $OPT(i - 1, v)$ 。如果它恰好使用 i 个边并以 (v, w) 开始，则剩余路径最多使用 $i - 1$ 个边。选择第一条边 (v, w) 后的成本为 $c_{vw} + OPT(i - 1, w)$ 。因此，

$$OPT(i, v) = \min \left\{ OPT(i - 1, v), \min_{(v,w) \in E} (c_{vw} + OPT(i - 1, w)) \right\}.$$

结论 这个递推式是贝尔曼-福特动态规划的基础。

为什么 n - 1 轮就足够了 (Why n - 1 Rounds Are Sufficient)

如果不存在相关的负成本循环，则每个有限最短距离都通过简单路径实现。图中的简单路径，其中 $n = |V|$ 顶点最多有 $n - 1$ 条边。因此， $OPT(n - 1, v)$ 是从 v 到 t 的真正最短距离，如果从 v 无法到达 t，则 $OPT(n - 1, v)$ 是 ∞ 。轮次的解释 第 1 轮允许最多有 1 个边缘的路径。第 2 轮允许最多有 2 条边的路径。第 i 轮允许路径最多有 i 个边。第 n - 1 轮考虑了所有可能的简单路径。

DP 表实现 (DP Table Implementation)

让 $M[i, v]$ 存储使用最多 i 个边从 v 到 t 的最短距离。 $M[0, v] = \infty$ 对所有 $v \neq t$, $M[0, t] = 0$.

对于每个 $i = 1, 2, \dots, n - 1$, 首先复制前一行, 然后松弛每条边:

$$M[i, v] \leftarrow M[i - 1, v], \quad M[i, v] \leftarrow \min\{M[i, v], M[i - 1, w] + c_{vw}\}.$$

最短路径 (G, t) : 对于每个节点 v : $M[0, v] = \infty$ $M[0, t] = 0$

对 $i = 1, \dots, n - 1$: 对于每个节点 v : $M[i, v] = M[i - 1, v]$

对于每条边 (v, w): $M[i, v] = \min(M[i, v], M[i - 1, w] + c[v, w])$

复杂度分析 (Complexity Analysis)

让 $n = |V|, m = |E|$.

有 $n - 1$ 轮。在每一轮中, 算法复制 n 个条目并扫描所有 m 个边。边缘松弛功为 $\Theta(mn)$.

该表包含 $n \times n$ 个条目, 因此空间复杂度 (Space Complexity) 为 $\Theta(n^2)$ 。

路径重构 当 $M[i, v]$ 通过边 (v, w) 改进时, 记录 w 作为 v 的后继者。

实际改进：单距离阵列 (Practical Improvement: One Distance Array)

完整的 DP 表在概念上很有用, 但并不总是必需的。维护一个数组 $M[v]$, 其中 $M[v]$ 是当前已知的从 v 到 t 的最佳距离, 并维护后继 $[v]$ 以恢复路径。如果从 v 到 t 的最佳路径经过邻居 w , 则

$$v \rightarrow w \rightarrow \dots \rightarrow t.$$

每当 $M[v] > M[w] + c_{vw}$ 时, 执行更新 $M[v] \leftarrow M[w] + c_{vw}$ 。

内存使用 内存变为 $O(m + n)$: 图形为 $O(m)$, 距离和后继为 $O(n)$ 。

高效的 Bellman-Ford 实施 (Efficient Bellman-Ford Implementation)

如果 $M[w]$ 在上一轮没有改变, 那么进入 w 的边无法通过 w 改进其起始节点。

基于推送的最短路径算法:

```
for each vertex v:
    M[v] = infinity
    successor[v] = null
M[t] = 0
updated = {t}
for i = 1, ..., n - 1:
    nextUpdated = empty set
    for each w in updated:
        for each incoming edge (v, w):
            if M[v] > M[w] + c[v, w]:
                M[v] = M[w] + c[v, w]
                successor[v] = w
                add v to nextUpdated
    if nextUpdated is empty:
        stop
    updated = nextUpdated
```

性能 最坏情况 (Worst-Case) 下的时间仍然是 $O(mn)$, 但在实践中提前停止和选择性扫描通常要快得多。

正确性直觉 (Correctness Intuition)

DP 不变量: 对于表格版本, 计算第 i 行后, 对每个顶点 v 都有 $M[i, v] = \text{OPT}(i, v)$ 。

第 1 行表示最多有 1 个边的路径。第 2 行说明最多有 2 条边的路径。第 i 行表示最多具有 i 个边的路径。每个有限值对应一条实际路径; 该算法并没有发明人为的距离。结论 经过 $n - 1$ 轮后, 如果不存在相关的负成本循环, 则所有简单路径均已被考虑, 并且最短距离是正确的。

从最短路径到路由 (From Shortest Paths to Routing)

在通信网络中,

节点 $\approx$ 路由器, 边缘 $\approx$ 直接通信链路,

边缘成本 $\approx$ 延迟、拥塞或路由成本。路由问题问: 对于每个目的地, 路由器应该将该数据包转发到哪个邻居? 这正是一个最短路径问题。Dijkstra 算法需要全局网络拓扑。Bellman-Ford 可以仅使用本地邻居信息来工作。

结论 Bellman-Ford 天然适合分布式路由。

距离矢量协议 (Distance Vector Protocol)

每个路由器维护一个到所有目的地的估计最短路径距离的向量。对于每个目的地 t , 路由器 v 存储: 从 v 到 t 的估计距离; 到 t 的路由上的下一跳邻居。每个路由器定期与其邻居共享其距离向量。如果路由器 v 从邻居 w 得知 w 到 t 的距离为 $D_w(t)$, 则 v 可以更新

$$D_v(t) \leftarrow \min\{D_v(t), c_{vw} + D_w(t)\}.$$

这里 c_{vw} 是 v 直接向邻居 w 发送流量的成本。通过谣言进行路由 每个路由器都不知道整个网络。它只知道其邻居的主张。

距离矢量路由的异步性质（Asynchronous Nature of Distance Vector Routing）

在实际网络中，路由器不会完美同步更新。它们并不同时执行第 1 轮、第 2 轮、第 3 轮。相反，更新是异步的。当路由器的距离估计发生变化时，它就会变得活跃并最终通知其邻居。这些邻居可以更新自己的估计并通知其他人。

只要边成本保持稳定、相关区域不存在可达的负成本环，并且每条松弛边都被公平且反复地处理（更新信息最终能够传播），异步 Bellman-Ford 仍会收敛到正确的最短距离。若存在负成本环，则有限最短距离不存在；若更新长期丢失或某些边不再被处理，也不能保证收敛。

计数到无穷大问题（The Counting-to-Infinity Problem）

当链路发生故障时，距离矢量路由可能会表现得很糟糕。最初，三个节点满足 $s \leftrightarrow v \rightarrow t$ ，每个所示的链路成本为 1。因此 $Dv(t) = 1, Ds(t) = 2$ 。

现在假设链接 $v \rightarrow t$ 被删除。理想情况下，两个路由器都应该意识到 t 不可达。但 s 仍可能公布其旧距离 $Ds(t) = 2$ 。然后 $Dv(t) = 1 + Ds(t) = 3, Ds(t) = 1 + Dv(t) = 4$,

然后 v 更新为 5， s 更新为 6，依此类推。结论 路由器在环路中相互路由。这就是计数到无穷大的问题。

为什么会出现计数到无穷大的情况？（Why Does Counting-to-Infinity Happen?）

距离矢量协议仅存储有限的信息：

到每个目的地的距离；每个目的地的下一跳。

他们不存储完整路径。因此，路由器可能认为邻居具有有效的路由，即使该邻居的路由实际上取决于路由器本身。在示例中， s 相信它可以通过 v 到达 t ，而 v 相信它可以通过 s 到达 t 。环路源 两台路由器都不知道完整的路由，因此都无法立即检测到环路。

路径向量（Path Vector）和链路状态（Link State）路由（Path Vector and Link State Routing）

距离向量：存储距离和下一跳。路径向量：存储更多的路径信息，通常是沿路线的节点或自治系统的序列。链路状态：收集网络拓扑，然后运行最短路径算法，通常是 Dijkstra 算法。

避免环路 路径向量可以拒绝已包含路由器本身的路径。链路状态避免计数到无穷大，因为路由器从更完整的网络视图计算路由。

示例：BGP 通常与路径向量路由相关联，而 OSPF 通常与链路状态路由相关联。这些协议减少了路由循环，但需要更多的存储、通信或全局信息。

距离向量 vs. 链路状态 vs. 路径向量（Distance Vector vs. Link State vs. Path Vector）

协议类型 存储的信息 主要算法思想 主要问题 距离向量 距离 + 下一跳 Bellman-Ford 计数到无穷大 链路状态 网络拓扑 Dijkstra 需要更多全局信息 路径向量 路径或路由向量 按路径进行环路检测 存储和策略复杂性

距离向量简单且分布式，但可能会受到收敛速度慢和路由环路的影响。链路状态提供了更好的全局一致性，但需要路由器维护更多的拓扑信息。Path Vector 存储路径信息，有助于避免环路并支持基于策略的路由。

总结（Summary）

负边权重使最短路径变得更加困难。当存在负边时，Dijkstra 算法可能会失败。向所有边添加常量不会保留最短路径。负成本循环可能会使最短路径变得不确定。如果没有相关的负成本循环，最短路径可以很简单并且最多使用 $n - 1$ 条边。Bellman-Ford 遵循边数上的 DP 递归。其标准运行时间为 $O(mn)$ ，实际距离存储为 $O(n)$ 。贝尔曼-福特自然而然地导致了距离矢量路由。距离向量是分布式的，但可能会受到计数到无穷大的影响。路径向量和链路状态路由存储更多信息以避免这些问题。

外卖 Bellman-Ford 不仅仅是一种针对负边沿的算法；它也是一种针对负边沿的算法。它也是分布式最短路径计算的模型。

第14周 网络流：最大流、最小割与增广路径（Network Flow: Maximum Flow, Minimum Cut, and Augmenting Paths）

为什么要研究网络流？（Why Study Network Flow?）

网络流是通过具有容量（Capacity）约束的有向网络发送资源的通用数学模型。交通网络：铁路、公路、物流。通信网络：数据包和带宽。匹配问题：将工人分配给工作或将学生分配给项目。航空公司调度和项目选择。图像分割。网络可靠性、连接性和分布式计算。

关键点 网络流量不仅仅是一个孤立的问题。它是一个建模框架，允许用同一种语言表达许多不同的优化问题。

两个核心问题（Two Central Problems）

最大流（Maximum Flow）量从 s 到 t 可以发送多少流量？

最小割（Minimum Cut）将 s 与 t 分开的最便宜的方法是什么？

主要定理，非正式的

最大流量值=最小割容量

可通过网络发送的流量总量受到分隔源与汇点的最窄瓶颈的限制。

流网络 (Flow Network) 定义 (Flow Network Definition)

流网络是一个有向图 (Directed Graph) $G = (V, E)$

它包含指定的源点 s 、汇点 t ，以及每条边 $e \in E$ 上的容量 $c(e) \geq 0$ 。容量 $c(e)$ 是边 e 可承载的最大流量。边是有方向的，流通常沿边的方向传输。**示例**: $s \rightarrow u \rightarrow t$ ，其中 $c(s, u) = 10$ 、 $c(u, t) = 7$ 。

通过该路径的最大可能流量为 7，因为第二条边是瓶颈。

(s)-(t) 切割的定义 (Definition of an (s)-(t) Cut)

(s)-(t) 割是一个划分 (A, B) ，使得

$$s \in A, \quad t \in B, \quad A \cup B = V, \quad A \cap B = \emptyset.$$

A 是源端。B 是汇点一侧。从 s 到 t 的每条路径必须至少从 A 到 B 交叉一次。

$$\text{cap}(A, B) = \sum_{\substack{e=(u,v) \in E \\ u \in A, v \in B}} c(e).$$

仅计算从 A 到 B 的边。从 B 回到 A 的边不计算在内。

割容量示例 (Example of Cut Capacity)

假设从 A 到 B 的边是

(s, u), (v, t), (x, t)

有能力 $c(s, u) = 10, c(v, t) = 5, c(x, t) = 15$ 。

于是 $\text{cap}(A, B) = 10 + 5 + 15 = 30$ 。

此割容量为 30。任何可行的 (s)-(t) 流的值必须最多为 30。割容量 30 并不自动意味着存在价值 30 的流量。为了证明最优性，我们需要具有相同值的流和割。

最小割问题 (Minimum Cut Problem)

最小割问题为

$$\min_{(A,B)} \text{cap}(A, B),$$

其中 (A, B) 遍历所有有效的 (s, t) 割。

一个网络可以有許多不同的切口。不同的切割可以具有不同的容量。最小割确定网络中最紧的瓶颈。它给出了最大流量的最佳可能上限。

直觉 如果我们删除从 A 到 B 的所有边，那么 s 就会与 t 断开。最小切割是断开源头和汇点的最便宜的方法。

流的定义 (Definition of a Flow)

(s)-(t) 流是一个函数 $f: E \rightarrow \mathbb{R}_{\geq 0}$

满足两个条件。容量限制

$$0 \leq f(e) \leq c(e), \quad \forall e \in E.$$

流守恒约束:

$$\sum_{e=(u,v) \in E} f(e) = \sum_{e=(v,w) \in E} f(e), \quad \forall v \in V \setminus \{s, t\}.$$

中间顶点不能创建或消耗流：任何进入的也必须离开。源 s 产生流量，汇 t 吸收流量。

流的价值 (Value of a Flow)

流的值定义为源点的净流出量:

$$v(f) = \sum_{e=(s,w) \in E} f(e) - \sum_{e=(u,s) \in E} f(e).$$

通过流守恒，它也等于汇点的净流入量。若模型额外假设没有边进入源点，则上式可简化为源点所有出边流量之和。最大流问题要求最大化 $v(f)$ 。

示例 如果源有传出流 4、6、10，则 $v(f) = 4 + 6 + 10 = 20$ 。

可行流 (Feasible Flow) 程示例 (Feasible Flow Example)

考虑小型网络

$s \rightarrow a, s \rightarrow b, a \rightarrow t, b \rightarrow t$ 。

假设 $f(s, a) = 5, f(s, b) = 3, f(a, t) = 5, f(b, t) = 3$ 。在 a 处：流入 = 5，流出 = 5。在 b 处：流入 = 3，流出 = 3。

因此，如果所有边缘流满足其容量约束，则该流是可行的。流量值为 $v(f) = 5 + 3 = 8$ 。

最大流量问题 (Maximum Flow Problem)

最大流问题是 $\max_f v(f)$

并满足

$$0 \leq f(e) \leq c(e), \quad \forall e \in E,$$

以及

$$\sum_{e=(u,v) \in E} f(e) = \sum_{e=(v,w) \in E} f(e), \quad \forall v \in V \setminus \{s, t\}.$$

限制是局部的。目标是全局性的。问题是找到从 s 到 t 的最大可行流量。

流值引理 (Flow Value Lemma)

引理 对于任何可行流 f 和任何 (s, t) 割 (A, B) ,

$$v(f) = \sum_{\substack{e=(u,v) \in E \\ u \in A, v \in B}} f(e) - \sum_{\substack{e=(u,v) \in E \\ u \in B, v \in A}} f(e).$$

左侧是源产生的总流量。右侧是穿过切口的净流量。从 A 到 B 的流量为正计数。从 B 到 A 的流量为负数。如果我们围绕 A 绘制边界，则逃离该边界的净量必须等于源创建的量。

流量值引理背后的直觉 (Intuition Behind the Flow Value Lemma)

证明依赖于内部取消。对于内部边缘 $u \rightarrow v, u, v \in A$,

对 A 中的所有顶点求和时：该边被计数为从 u 传出，同一边被计数为进入 v 一次，这两个贡献抵消。因此，仅保留穿过 A 边界的边。所有中间顶点都满足守恒，因此只有源贡献非零净流。

流量值引理的证明 (Proof of the Flow Value Lemma)

对于每个顶点 $v \in A$ ，定义净流出量

$$\text{excess}(v) = \sum_{e=(v,w) \in E} f(e) - \sum_{e=(u,v) \in E} f(e).$$

对每个 $v \in A \setminus \{s\}$ ，流守恒给出 $\text{excess}(v) = 0$ 。因此 $v(f) = \sum_{v \in A} \text{excess}(v)$ 。

对右侧求和时，A 内部边的贡献相互抵消，只留下穿过割的边，从而得到流值引理。

弱对偶性 (Weak Duality)

弱对偶性 (Weak Duality) 对于任何可行流 f 和任何 (s)-(t) 割 (A, B) ， $v(f) \leq \text{cap}(A, B)$ 。

割断给出了每个可行流的上限。流量不能超过从源端到汇端交叉的边的总容量。这就是为什么切割对于证明上限很有用。

弱对偶性证明 (Proof of Weak Duality)

从流值引理和流量非负性出发，

$$\begin{aligned} v(f) &= \sum_{\substack{e=(u,v) \in E \\ u \in A, v \in B}} f(e) - \sum_{\substack{e=(u,v) \in E \\ u \in B, v \in A}} f(e) \\ &\leq \sum_{\substack{e=(u,v) \in E \\ u \in A, v \in B}} f(e) \\ &\leq \sum_{\substack{e=(u,v) \in E \\ u \in A, v \in B}} c(e) = \text{cap}(A, B). \end{aligned}$$

最优证书 (Certificate of Optimality)

如果存在流 f 和割 (A, B) ，使得 $v(f) = \text{cap}(A, B)$ ，

那么：f是最大流量。(A,B)是最小割。弱对偶性表示任何流都不能超过任何割。如果一个流量达到了一次切割的价值，那么两者都无法改善。**示例** 如果

$$v(f) = 28 \quad \text{且} \quad \text{cap}(A, B) = 28,$$

那么最大流量值正好是 28。

朴素贪婪算法 (Naive Greedy Algorithm)

1. 从所有边的 $f(e) = 0$ 开始。
2. 找到一条边缘不饱和的(s)-(t)路径。
3. 沿该路径推动尽可能多的流量。
4. 重复直到不存在这样的路径。

为什么它看起来合理 它总是会增加流量。它尊重能力。它很容易实现。

警告 这种贪婪策略是不够的，因为它无法撤销之前的错误选择。

为什么天真的贪婪会失败 (Why Naive Greedy Can Fail)

贪心算法（Greedy Algorithm）可能会通过中间边缘发送过多的流量，而该中间边缘随后会成为瓶颈。然后：一些边缘变得饱和，不再出现可用的前向 (s)-t) 路径，如果重新路由由先前的流，可能仍然存在更好的解决方案。

教学要点 陷入原始图表中并不意味着电流达到最大。

需要撤消流程（Need to Undo Flow）

最大流算法需要两个能力：

1. 向前发送附加流。
2. 取消或重新路由由先前发送的流。

关键想法 为了修复之前的选择，我们在称为残差图的新图中添加了相反的方向。

前向剩余边缘意味着我们仍然可以发送更多流量。向后剩余边缘意味着我们可以撤消之前发送的一些流。

剩余容量（Residual Capacity）

对于原始边 $e = (u, v)$

根据容量 $c(e)$ 和当前流 $f(e)$ ，定义前向与后向残余容量： $c_f(u, v) = c(e) - f(e)$

和 $c_f(v, u) = f(e)$ 。

$c(e) - f(e)$ 是还可以从 u 发送到 v 的流量。 $f(e)$ 是可以取消从 v 回到 u 的流量。**示例** 如果 $c(e) = 17$ 且 $f(e) = 6$ ，则 $c_f(u, v) = 11$ ， $c_f(v, u) = 6$ 。

残差图（Residual Graph）

残差图为 $G_f = (V, E_f)$ 。

它包含：如果 $f(u, v) < c(u, v)$ ，则前向残差边 $u \rightarrow v$ ；如果 $f(u, v) > 0$ ，则后向残差边 $v \rightarrow u$ 。正式地， $E_f = \{e : c_f(e) > 0\}$ 。

若原图同时含有 (u, v) 和 (v, u) ，应把“原边产生的前向残余弧”和“反向撤销弧”视为可区分的残余弧，或将同方向的残余容量相加，避免在集合表示中错误合并。

残差图表示电流的所有合法局部变化。每次增强后它都会发生变化。它是 Ford-Fulkerson 的中央数据结构。

增强路径（Augmenting Path）

增广路径（Augmenting Path）是残差图 G_f 中从 s 到 t 的简单路径（Simple Path）。其瓶颈容量（Bottleneck Capacity）为

$$\text{bottleneck}(P) = \min_{e \in P} c_f(e).$$

瓶颈是沿着整个路径可以推动的最大流量。路径上的每条边都至少可以支持这么多的变化。增强后，流量值正好增加了瓶颈量： $v(f') = v(f) + \text{bottleneck}(P)$ 。

增广操作（Augmentation Operation）

让 $b = \text{bottleneck}(P)$ 。

对于增广路径上的每条边：如果该边是原图中的前向边，则将其流量增加 b ；如果该边是后向残差边，则将对应的原始边上的流量减少 b 。规则： $f(e) \leftarrow f(e) + b$ 表示前向边缘， $f(e) \leftarrow f(e) - b$ 表示反向边缘。前向边缘添加新流，后向边缘取消旧流，并且所得流仍然可行。

Ford-Fulkerson 算法（Ford-Fulkerson Algorithm）

福特-富尔克森(G, s, t, c)

1. 对每条边 $e \in E$ 初始化 $f(e) = 0$ 。
 2. 构造残差图 G_f 。
 3. 当 G_f 中存在 (s, t) 路径 P 时：
 设 $b = \text{瓶颈}(P)$ 。将 f 沿 P 增广 b 。更新 G_f 。
 4. 返回 f 。
- 该算法反复改进电流。唯一未指定的选择是使用哪个增广路径。

增广路径定理（Augmenting Path Theorem）

定理 当且仅当残差图 G_f 中不存在增广路径时，流 f 才是最大流。

如果存在增广路径，则流量可以增加，因此 f 不是最大值。如果没有增广路径，则没有剩余改进可以将额外的流从 s 发送到 t 。该定理将算法停止条件与最优性联系起来。

最大流最小割定理（Max-Flow Min-Cut Theorem）

最大流最小割定理（Max-Flow Min-Cut Theorem）

$$\max_f v(f) = \min_{(A, B)} \text{cap}(A, B).$$

最大流量等于最小切割的容量。最大流量和最小割量是同一数量的两个视图。该定理提供了算法和最优性证明。

证明结构（Proof Structure）

通过证明以下陈述是等价的来证明该定理：

1. 存在割 (A, B) , 使得 $v(f) = \text{cap}(A, B)$ 。
2. f 是最大流量。
3. G_f 中没有增广路径。

(1 $\Rightarrow$ 2): 由弱对偶性得出。(2 $\Rightarrow$ 3): 如果存在增广路径, 则流量可以增加。(3 $\Rightarrow$ 1): 从残差图构造一个切割。

构建剪切 (Constructing the Cut)

假设 G_f 中没有增广路径。定义 $A = \{v \in V : v \text{ 在 } G_f \text{ 中可由 } s \text{ 到达}\}$ 。

和 $B = V - A$ 。

然后:

$$s \in A \quad \text{由定义可知。}$$

$t \notin A$, 否则存在增广路径。因此, (A, B) 是有效的 $(s) - (t)$ 切割。该切割是直接根据最终残差图构建的。

为什么构造切工很紧 (Why the Constructed Cut Is Tight)

从 A 到 B 的每条边都是饱和的。如果 $e = (u, v)$, 其中 $u \in A$ 且 $v \in B$, 并且

$$f(e) < c(e),$$

那么就会有从 u 到 v 的前向残差边缘。由于 u 从 s 可达, 因此 v 也可达, 这是一个矛盾。因此, $f(e) = c(e)$

对于从 A 到 B 的每条边。此外, 从 B 到 A 的每条边的流量为零: 如果这样的边具有正流量, 则残差图将包含从 A 到 B 的后向边, 这又与 A 的定义相矛盾。

完成证明 (Completing the Proof)

由流值引理,

$$v(f) = \sum_{\substack{e=(u,v) \in E \\ u \in A, v \in B}} f(e) - \sum_{\substack{e=(u,v) \in E \\ u \in B, v \in A}} f(e).$$

从 A 到 B 的边全部饱和, 而从 B 到 A 的边流量为零。因此

$$v(f) = \sum_{\substack{e=(u,v) \in E \\ u \in A, v \in B}} c(e) = \text{cap}(A, B).$$

因此流量值等于切割能力。通过弱对偶性, 流量最大, 切割最小。

Ford-Fulkerson 的运行时间 (Running Time of Ford-Fulkerson)

假设所有容量都是整数。每次增广都会使流量值至少增加 1, 因为瓶颈容量是正整数。如果最大流值为 $v(f^*)$, 则增广次数至多为 $v(f^*)$ 。

如果可以在 $O(m)$ 时间内找到一条增广路径, 则运行时间为 $O(m v(f^*))$ 。

这是伪多项式。这取决于最大流量的数值。如果容量很大, 算法可能会很慢。

整数性定理 (Integrality Theorem)

完整性定理 如果所有容量都是整数, 则存在最大流, 其中每条边流 $f(e)$ 都是整数。

Ford-Fulkerson 从整数流开始。剩余容量 (Residual Capacity) 仍为整数。每个瓶颈值都是一个整数。每次增强都保持完整性。因此最终的最大流量是积分的。

教学要点 这对于二分匹配等组合应用很重要。

为什么路径选择很重要 (Why Path Selection Matters)

Ford-Fulkerson 没有指定选择哪条增广路径。这很重要, 因为: 有些选择会使算法很快, 有些选择则会使算法非常慢; 当容量为无理数 (irrational capacities) 时, 任意选择增广路径甚至可能导致 Ford-Fulkerson 不终止。

目标 选择增广路径, 以便: 1、每条路径都能高效找到; 2.增广数量少。

错误的增强路径 (Bad Augmenting Paths)

典型的不良情况有两条大容量路由, 但算法会重复选择经过小容量边缘的路径。然后: 每次增强仅增加少量流量, 后来的增强可能会使用反向边缘撤消先前的选择, 该算法仍然正确, 但可能需要多次迭代。

教学点的正确性并不自动意味着效率。

更好的增强路径策略 (Strategies for Better Augmenting Paths)

1. 最胖路径 选择瓶颈容量最大的路径。
每次发送尽可能多的流量。找到它需要额外的工作。2.足够大的瓶颈 不需要最大的瓶颈。仅要求瓶颈至少有一个阈值。导致容量扩展。
2. 最少边数 选择边数最少的路径。
可以通过BFS找到。导出最短增广路径 (Shortest Augmenting Path) 方法。

容量扩展主要思想 (Capacity Scaling Main Idea)

维持缩放参数 Δ 。将阈值残余图定义为

$$G_f(\Delta) = (V, \{e : c_f(e) \geq \Delta\}).$$

那么 $G_f(\Delta)$ 中的任何增广路径都具有至少为 Δ 的瓶颈容量。较大的 Δ 只允许较大的残余边；随着 Δ 减小，较小的残余边逐渐被纳入，从而避免过早花费时间处理很小的增广。

容量扩展算法 (Capacity Scaling Algorithm)

1. 初始化 $f(e) = 0, \forall e \in E$. 2. 令 $C = \max_{e \in E} c(e)$ ，并令 Δ 为不超过 C 的最大 2 的幂。

```
while Delta >= 1:
    构造 G_f(Delta)
    while G_f(Delta) 中存在 s-t 路径 P:
        沿 P 增广
        更新残余图
    Delta <- Delta / 2
return f
```

容量扩展的正确性 (Correctness of Capacity Scaling)

假设所有容量都是 1 到 C 之间的整数。两个不变量：

- Δ 始终是 2 的幂。
- 所有流量和剩余容量保持整数。当 $\Delta = 1$ 时, $G_f(1) = G_f$.

因为每个正整数剩余容量至少为 1。因此，当 $\Delta = 1$ 阶段结束时，完整残余图中不存在增广路径。根据增广路径定理，所得流量最大。

缩放阶段数 (Number of Scaling Phases)

有 $O(\log C)$ 个缩放阶段。初始时 $\Delta \leq C$ 。

每个阶段将 Δ 除以 2。当 $\Delta < 1$ 时算法停止，因此阶段数为 $O(\log C)$ 。

容量扩展的关键引理 (Key Lemma for Capacity Scaling)

关键引理：在 Δ 缩放阶段结束时，若当前流为 f ，则 $\text{val}(f^*) \leq v(f) + m\Delta$ ，其中 $m = |E|$ 。

Δ 阶段结束后，所有剩余改进都必须使用残余容量小于 Δ 的边。由于至多有 m 条边，因此距离最优值的差距至多为 $m\Delta$ 。

关键引理的证明思路 (Proof Idea of the Key Lemma)

设 A 为 $G_f(\Delta)$ 中从 s 可达的顶点集。在 Δ 阶段结束时，在 $G_f(\Delta)$ 中不存在 s - t 路径，因此 $t \notin A$ 。

因此 $(A, V \setminus A)$ 是一个 (s, t) 割。对于从 A 到 B 的任意边，其残余容量都小于 Δ ；否则，它会出现于 $G_f(\Delta)$ 中并使终点可达。因此 $f(e) > c(e) - \Delta$ 。对于从 B 到 A 的边，流量小于 Δ ；否则会存在残余容量至少为 Δ 的反向边。由流值引理， $v(f) \geq \text{cap}(A, B) - m\Delta$ ，

所以 $\text{cap}(A, B) \leq v(f) + m\Delta$ 。

由于最大流量至多是任何切割的容量，

$\text{val}(f^*) \leq v(f) + m\Delta$ 。

每个扩展阶段的增强 (Augmentations per Scaling Phase)

每个 Δ 缩放阶段最多有 $2m$ 次增广。在当前 Δ 阶段开始时，前一阶段的参数为 2Δ 。根据关键引理，最大流值至多为 $v(f) + m(2\Delta)$ ，所以剩余可增加的流量至多为 $2m\Delta$ 。

在当前 Δ 相期间，每次增强都会使流量至少增加 Δ 。因此，增强次数最多为 $\frac{2m\Delta}{\Delta} = 2m$ 。

扩容运行时间 (Running Time of Capacity Scaling)

结合之前的结果：相数： $O(\log C)$

每阶段的增强数量： $O(m)$

是时候找到一条增广路径了： $O(m)$

因此，总运行时间为 $O(m^2 \log C)$ 。

这是图大小的多项式和容量界限的对数。

最短增强路径策略 (Shortest Augmenting Path Strategy)

不要选择具有较大瓶颈容量的路径，而应选择具有最少边数的路径。这可以通过广度优先搜索 (Breadth-First Search, BFS) 找到：

$P \leftarrow \text{BFS}(G_f)$ 。

澄清：“最短”意味着最少的边缘。这并不意味着最小容量。它并不意味着最小总重量。这种策略催生了 Edmonds-Karp 算法。

最短增广路径算法 (Shortest Augmenting Path Algorithm)

1. 初始化 $f(e) = 0, \forall e \in E$. 2. 构造残余图 G_f 。

- 当 G_f 中存在 (s, t) 路径时：
使用 BFS 寻找边数最少的增广路径 P 。沿 P 增广 f 。更新 G_f 。
- 返回 f 。

BFS给出了系统的路径选择规则。这避免了任意的错误选择。该方法仍然简单且易于实现。它就是 Edmonds-Karp 算法，增广次数为 $O(nm)$ ，每次 BFS 需要 $O(m)$ 时间，因此总运行时间为 $O(nm^2)$ 。

增强路径策略的比较 (Comparison of Augmenting Path Strategies)

方法	路径选择规则	运行时间或特点
任意 Ford-Fulkerson	任意增广路径	简单，但可能很慢或不终止
容量缩放	瓶颈至少为 Δ	$O(m^2 \log C)$
Edmonds-Karp	BFS 选择最少边数的路径	$O(nm^2)$

容量扩展重点关注瓶颈大小。最短增广路径关注残差图中的距离。两者都是提高福特-福克森公司效率的方法。

最终总结 (Final Summary)

- 流网络对通过有能力的有向边的运动进行建模。
- 可行的流量满足容量和流守恒约束。
- 切口将 s 与 t 分开，其容量上限为每个流。
- 弱对偶性给出 $v(f) \leq \text{cap}(A, B)$ 。
- 如果流量和割量具有相同的值，则两者都是最优的。
- 残差图允许添加流和撤消先前的流。
- Ford-Fulkerson 沿着残差 (s,t) 路径重复增广。
 - 没有增广路径意味着最大流量。
- 最大流量值等于最小切割能力。
- 仔细选择增广路径对于提高效率至关重要。

残差图 $\Rightarrow$ 增广路径 $\Rightarrow$ 最大流 $\Leftrightarrow$ 最小割

最终消息网络流是一个模型、一个算法框架和一种证明技术。理解该主题的关键是理解残差图和割图。

第15周 网络流的应用与扩展 (Network Flow: Applications and Extensions)

无向图 (Undirected Graph) 中的匹配 (Matching in an Undirected Graph)

给定一个无向图 $G = (V, E)$,

匹配是一个子集 $M \subseteq E$

使得每个顶点最多与 M 中的一条边相关。没有两条选定的边共享端点。顶点可以是不匹配的。最大基数匹配具有尽可能多的边。

小示例 如果选择 (a, b) ，则 a 和 b 都不能被其他选定的边使用。

二分图 (Bipartite Graph) 和二分匹配 (Bipartite Graphs and Bipartite Matching)

一个图是二分图，如果 $G = (L \cup R, E)$,

其中每条边将 L 中的一个顶点连接到 R 中的一个顶点。 L 内部没有边， R 内部也没有边。二分匹配只选择 L 与 R 之间的边，并且每个顶点最多使用一次。这种结构使得最大匹配可以归约为最大流 (Maximum Flow)。应用程序包括学生到项目、工人到工作、广告商到广告位、患者到兼容的捐赠者。

最大二分匹配 (Maximum Bipartite Matching)

输入：二分图 $G = (L \cup R, E)$ 。输出：基数最大的匹配。大小为 3 的匹配表示选择了 3 对顶点。最大匹配是基数最大的有效边集。

重要说明 即使 $|L| = |R|$ ，完美匹配 (Perfect Matching) 可能不存在。边缘结构很重要。

例如，如果几个左侧顶点的可能邻居太少，则并非所有顶点都可以匹配。

减少二分匹配到最大流 (Reducing Bipartite Matching to Maximum Flow)

根据 $G = (L \cup R, E)$ 构造有向网络 (Flow Network) G' 。添加源点 s 和汇点 t 。对每个 $u \in L$ 添加容量为 1 的边 (s, u) ；将每条原始二分边从 L 定向到 R ；对每个 $v \in R$ 添加容量为 1 的边 (v, t) 。求匹配时可令中间边容量为 1；证明 Hall 定理时令其容量为 ∞ 。一个单位的流

$s \rightarrow u \rightarrow v \rightarrow t$

对应于将 u 与 v 匹配。

为什么要进行施工 (Why the Construction Works)

主要定理 G' 中最大流的值 = G 中最大匹配的大小。

$s \rightarrow L$ 上的容量 1 表示每个左侧顶点最多使用一次。 $R \rightarrow t$ 上的容量 1 表示每个右顶点最多使用一次。中间边缘代表允许的匹配；当端点容量为 1 时，它们的容量永远不会成为瓶颈。因此：一个可行的积分流对应一个有效的匹配。一个匹配对应一个可行的流程。这是一对一的建模思路，不仅仅是巧合。

证明方向 1——匹配意味着流程 (Proof Direction 1 — Matching Implies Flow)

假设 G 有一个大小为 k 的匹配 M 。对于每个匹配的边 $(u, v) \in M, u \in L, v \in R$,

在 $s \rightarrow u \rightarrow v \rightarrow t$ 上发送 1 个流量单位。

因为 M 是匹配，所以没有左顶点被使用两次。没有右顶点被使用两次。因此，所有容量限制均得到满足。构造的流具有值 k 。

最大流量值 $\geq$ 匹配大小。

证明方向 2——流程意味着匹配 (Proof Direction 2 — Flow Implies Matching)

假设 G' 的最大流值为 k 。根据整数性定理，由于容量都是整数，因此存在整数最大流。在该网络中，每条边的流量可取为 0 或 1。构造 $M = \{(u, v) \in E : f(u, v) = 1, u \in L, v \in R\}$ 。

为什么 M 是匹配？每个 $u \in L$ 最多从 s 接收 1 个单位，每个 $v \in R$ 最多向 t 发送 1 个单位，因此没有顶点会被重复使用，且 $|M| = k$ 。

完美匹配 (Perfect Matching)

如果每个顶点恰好出现在一条匹配边中，则匹配 M 是完美匹配。在二分图 $G = (L \cup R, E)$ 中，完美匹配要求 $|L| = |R|$ 。

但这个条件并不充分，即使两边有相同数量的顶点， L 的某些子集在 R 中的邻居也可能太少。

霍尔定理所发现的正是这种可能合作伙伴的短缺。

邻居集 (Neighbor Sets)

对任意子集 $S \subseteq L$ ，定义

$$N(S) = \{v \in R : \exists u \in S, (u, v) \in E\}.$$

$N(S)$ 是与 S 中的顶点相邻的右侧顶点的集合。完美匹配的必要条件是

$$|N(S)| \geq |S|, \quad \forall S \subseteq L.$$

S 中的每个顶点必须与 $N(S)$ 中的不同顶点匹配。如果 3 个左顶点只有 2 个可能的邻居，则完美匹配是不可能的。

Hall 婚姻定理 (Hall's Marriage Theorem) (Hall's Marriage Theorem)

Hall 定理：令 $G = (L \cup R, E)$ 为二分图并假设 $|L| = |R|$ ，则 G 有完美匹配当且仅当

$$|N(S)| \geq |S|, \quad \forall S \subseteq L.$$

必要性： S 中的每个顶点都需要一个独特的匹配伙伴。充分性：重要；可以使用最大流最小割 (Minimum Cut) 来证明。

霍尔条件准确地描述了何时存在完美匹配。

用最大流证明霍尔定理 (Proving Hall's Theorem with Max-Flow)

构建相同的流网络进行二分匹配，从 L 到 R 的每个中间边分配容量 ∞ 。为了矛盾，假设不存在完美匹配。那么最大匹配尺寸小于

$|L|$ 。

因此最大流值小于 $|L|$ 。由最大流最小割定理，存在 (s, t) 割 (A, B) ，使得 $\text{cap}(A, B) < |L|$ 。

目标：使用此割图构造一个违反霍尔条件的集合 $S \subseteq L$ 。

剪切结构 (The Cut Structure)

定义 $L_A = L \cap A$ 、 $L_B = L \cap B$ 、 $R_A = R \cap A$ 。

其中 $s \in A$ 且 $t \in B$ 。对于有限容量的最小割， $\text{cap}(A, B) = |L_B| + |R_A|$ 。

边 $s \rightarrow L_B$ 穿过割，每条容量为 1；边 $R_A \rightarrow t$ 也穿过割，每条容量为 1。从 L_A 到 $R_B = R \cap B$ 的边容量为 ∞ ，因此有限容量的最小割不能包含这些边。这种割结构揭示了邻居不足。

为什么 $N(L_A) \subseteq R_A$ (Why $N(L_A) \subseteq R_A$)

命题： $N(L_A) \subseteq R_A$ 。

反设不成立，则存在 $x \in L_A$ 、 $y \in R_B$

边 $x \rightarrow y$ 。这条边从 A 穿过到 B 。但中间边缘的容量为无穷大。那么削减能力将是无限的，矛盾

但已知 $\text{cap}(A, B) < |L|$ ，矛盾。

因此 L_A 的所有邻居都必须位于 R_A 中。

完成霍尔定理的证明 (Finishing the Proof of Hall's Theorem)

由该割可得 $\text{cap}(A, B) = |L_B| + |R_A| < |L|$ 。

因此 $|R_A| < |L| - |L_B| = |L_A|$ 。

又因为 $N(L_A) \subseteq R_A$,

所以 $|N(L_A)| \leq |R_A| < |L_A|$ 。

令 $S = L_A$, 则 $|N(S)| < |S|$, 违反 Hall 条件。因此, 如果 Hall 条件成立, 就必定存在完美匹配。

二分匹配的运行时间 (Running Time for Bipartite Matching)

我们可以使用最大流来解决二分匹配。在构建的网络中, 一条增广路径 (Augmenting Path) 将匹配大小增加 1。使用基本的增广路径最大流算法, 每次搜索需要 $O(m + n)$ 时间, 并且最多有 $|M^*| \leq n$ 次增广: $O((m + n)|M^*|) \subseteq O((m + n)n)$ 。

更专业的 Hopcroft-Karp 算法可将运行时间改进为 $O(m\sqrt{n})$ 。

在二分匹配中, 容量通常为 1, 因此问题是特别结构化的。注意 非二分匹配更加困难, 需要诸如 Edmonds 的开花算法之类的算法。

边不相交路径 (Edge-Disjoint Paths)

如果两条路径不共享边, 则它们是边不相交的。它们可能共享顶点。问题给定一个有向图 (Directed Graph) $G = (V, E)$

和顶点 s, t, 找到边不相交 $s \rightarrow t$ 路径的最大数量。

动机: 通信网络、路由可靠性和容错。更多不相交的路径意味着更独立的信息传输方式。

边不相交路径 (Edge-Disjoint Paths) 作为最大流 (Edge-Disjoint Paths as Maximum Flow)

为每条边分配容量 1。计算最大 s-t 流量。

定理 边不相交的最大数量 $s \rightarrow t$ 路径 = 最大流值。

每条路径承载 1 个单位的流量。容量 1 防止两条路径使用相同的边。这是积分流代表离散对象的另一个例子。

证明——路径意味着流程 (Proof — Paths Imply Flow)

假设有 k 条边不相交的路径

$P_1, P_2, \dots, P_k$ 。

定义

$$f(e) = \begin{cases} 1, & e \text{ 被某条 } P_i \text{ 使用,} \\ 0, & \text{否则.} \end{cases}$$

由于路径是边不相交的, 因此没有边接收超过 1 的流量。满足容量约束。从 s 到 t, 每条路径贡献 1 个单位的流量。因此流量值为k。

证明——流程意味着路径 (Proof — Flow Implies Paths)

假设最大流量的值为 k。由于所有容量都是整数, 并且每条边的容量均为 1, 因此完整性定理给出了 0-1 的最大流。从 s 的出边开始, 流为 1。遵循流程 1 的边缘。根据流守恒 (Flow Conservation), 每当流进入中间顶点时, 流也必须离开它。继续直到达到t; 如果遇到有向循环, 则删除该循环并继续跟随正流边缘。这会提取一个 $s \rightarrow t$ 路径。减去该路径的单位流量并重复以获得 k 个边不相交的路径。这正是流分解定理的路径部分。

网络连接 (Network Connectivity)

一组边 $F \subseteq E$

如果每条 $s \rightarrow t$ 路径至少使用 F 中的一条边, 则断开 t 与 s 的连接。网络连通性问题 找到移除会破坏所有 $s \rightarrow t$ 路径的最小边数。

这是单位边容量的最小割问题。它衡量网络针对边缘故障的稳健程度。

Menger定理 (Menger's Theorem) (Menger's Theorem)

Menger定理 (Menger's Theorem) 的边缘版本

$\max\{\text{边不相交 } s \rightarrow t \text{ 路径}\} = \min\{\text{删除将 t 与 s 断开的边}\}$ 。

这是最大流最小割的直接结果。左侧: 有多少条独立的路线? 右侧: 必须删除多少条边才能破坏所有路线?

Menger定理 (Menger's Theorem) 的证明 (Proof of Menger's Theorem)

第一个方向: 如果 F 将 t 与 s 断开并且 $|F| = k$, 则每条 $s \rightarrow t$ 路径至少使用 F 中的一条边。边不相交的路径不能共享边, 因此最多可以有 k 条边不相交的路径。第二个方向: 将所有边容量设置为1。如果边不相交路径的最大数量为 k, 则 max-flow 值为 k。通过最大流最小割, 存在容量 k 的 s-t 割。从源侧到汇侧的切割边缘形成尺寸为 k 的断开组。因此两个数量相等。

无向图中的边不相交路径 (Edge-Disjoint Paths in Undirected Graphs)

现在考虑一个无向图 $G = (V, E)$ 。

目标: 找到边不相交的 s-t 路径的最大数量。这些路径没有方向, 但它们仍然不能共享无向边。自然变换将每个无向边 {u, v} 替换为两个有向边:

(u, v) 和 (v, u) 。

给两个容量 1。然而, 这种转变引入了一个微妙的问题。

反平行边问题 (The Antiparallel Edge Issue)

在有向变换中，两条路径可能出现边不相交：一种使用 (u, v) 。另一个使用 (v, u) 。但在原始无向图中，两者都使用相同的边 $\{u, v\}$ 。

问题 有向边不相交并不自动与无向边不相交相同。

我们需要一个引理来消除相反方向的同时流动。

反平行边的引理 (Lemma for Antiparallel Edges)

引理 对于每个相反对 $e = (u, v)$ 和 $e' = (v, u)$ ，某些最大流量满足

$$f(e) = 0 \quad \text{或} \quad f(e') = 0 \quad (\text{或两者都为零}).$$

证明思路：

如果两个方向都有正流，则令 $\delta = \min\{f(e), f(e')\}$ 。

将两个流量均减少 δ 。容量、 u 和 v 处的守恒以及总价值保持不变。重复此操作，直到没有相对的对在两个方向上承载流动。

通过最大流的无向边不相交路径 (Undirected Edge-Disjoint Paths via Max-Flow)

通过反平行边引理，将该问题归约为最大流是正确的。将每个无向边替换为两个相对的有向边。给每个有向边容量 1。计算从 s 到 t 的最大流量。在没有同时相反方向使用的情况下存在最大流量，因此该流量可以解释为原始图中边不相交的路径。

定理

边不相交 s - t 路径的最大数量 = 变换后网络中的最大流值。

多个源和多个汇点 (Multiple Sources and Multiple Sinks)

标准最大流有一个源 s 和一个汇 t 。真实的网络可能有多个来源 $s_1, s_2, \dots, s_p$

和多个汇点 $t_1, t_2, \dots, t_q$ 。

问题 最大化从所有源节点发送到所有宿节点的总流量。

示例包括向多个仓库发货的多个工厂、为多个客户提供服务的多个数据中心以及为多个需求 (Demand) 节点提供数据的多个供应节点。

超级源和超级汇 (Super Source and Super Sink)

将问题转换为普通的最大流问题。添加新的超级源 s 。添加一个新的超级汇点 t 。对于每个原始源 s_i ，添加边 (s, s_i) ；对于每个原始汇点 t_j ，添加边 (t_j, t) 。这些新边的容量可设为不会成为瓶颈的有限上界，例如原网络所有边容量之和 $U = \sum_{e \in E} c(e)$ 。

理论叙述中也常将其简写为 $+\infty$ 。然后求解从 s 到 t 的标准最大流量。无限容量的边缘不应成为瓶颈；所有真正的瓶颈仍然存在于原始网络中。

流程一一对应 (One-to-One Correspondence of Flows)

命题 原始多源/多汇网络中的流与改造后的单源/单汇网络中的流一一对应。

从原始到转换：将从超级源到每个原始源的流量添加等于其总输出流量。将每个原始汇点的流量添加到超级汇点，等于其总传入流量。从变换到原始：删除超级源和超级汇边缘。剩余的流是有效的多源/多宿流。

多源/多汇最大流量=变换网络中的普通最大流量。

供应和需求的流通 (Circulation with Supplies and Demands)

更通用的模型允许一些节点供应流量，一些节点需求流量，一些节点仅转运流量。给定 $G = (V, E)$ 、容量 $c(e) \geq 0$ 与节点需求 $d(v)$ ，循环流 (Circulation) 是满足以下条件的函数 $f(e)$ ： $0 \leq f(e) \leq c(e)$ ， $\forall e \in E$ 。

每个节点满足平衡条件： $\sum_{e \text{ 进入 } v} f(e) - \sum_{e \text{ 离开 } v} f(e) = d(v)$ 。

解释 d(v) (Interpreting d(v))

$d(v)$ 的符号决定了节点 v 的角色。如果 $d(v) > 0$ ，则 v 是需求节点：它需要净流入。如果 $d(v) < 0$ ，则 v 是供应节点：它提供净流出。如果 $d(v) = 0$ ，则 v 是转运节点：流入等于流出。可行性 (Feasibility) 要求总供给 (Supply) 等于总需求： $\sum_{v \in V} d(v) = 0$ 。

将循环流归约为最大流 (Reducing Circulation to Max-Flow)

添加新的源点 s 和新的汇点 t ，形成辅助网络 G' 。对于 $d(v) < 0$ 的每个供应节点，添加容量为 $-d(v)$ 的边 (s, v) 。对于 $d(v) > 0$ 的每个需求节点，添加容量为 $d(v)$ 的边 (v, t) 。保留所有原始边缘和容量。然后从 s 到 t 运行最大流 (Max-Flow) 算法。

流通可行性条件 (Feasibility Condition for Circulation)

定义总需求： $D = \sum_{v: d(v) > 0} d(v)$ 。

由于总供给等于总需求， $D = \sum_{v: d(v) < 0} -d(v)$ 。

可行性标准

G 有一个可行循环 $\iff G'$ 有一个值为 D 的最大流。

同样，离开新源 s 的所有边都必须饱和。当总供给等于总需求时，这也会使进入新汇点 t 的所有边缘饱和。释义：每一单位的供给都被发送，每一单位的需求都被满足。

循环流的整数性 (Integrality for Circulation)

整数性定理 (Integrality Theorem)：如果所有容量和需求都是整数，并且存在可行循环流，则存在整数值的可行循环流。

循环流问题可归约为最大流问题。当容量为整数时，最大流具有整数性。这很重要，因为许多应用需要离散单元：人员、工作、数据包、物品和任务。

流通切断条件 (Cut Condition for Circulation)

如果不存在可行循环流，则存在不可行性证书。令 $b(B) = \sum_{v \in B} d(v)$, $A = V \setminus B$.

某个违反条件的集合 $B \subseteq V$ 满足 $b(B) > c(A, B)$,

其中 $c(A, B)$ 是从 A 进入 B 的边的总容量。同样，在教学语言中：B 的需求 > B 的供给 + 从 A 到 B 的产能。

B 区需要的净流入量超出其所能接收的量。内部供应加上传入产能不足。这是最小切口阻塞的循环模拟。

统一视图 (Unifying View)

相同的建模模式反复出现。二分匹配：一个流量单位等于一个匹配对。边不相交路径：一单位流量等于一条独立路径。网络连接：最小切割等于要删除的最小边数。多个源/汇：超级源和超级汇将问题减少到标准最大流量。流通：供给和需求转换为源/汇边缘。

最大流 (Max-Flow) 是离散优化中的通用工具。

最后要点 (Final Takeaways)

最大流提供算法、证明技术和结构定理。涵盖的关键定理：通过最大流进行二分匹配。Hall婚姻定理 (Hall's Marriage Theorem)。Menger定理 (Menger's Theorem)。通过最大流量实现循环可行性。重要证明工具：整数性定理。流分解。最大流最小割定理 (Max-Flow Min-Cut Theorem)。

许多组合问题一旦表达为流网络就会变得更容易。